

Formalization of EqCat_∞ in Rocq

Jack Widman

June 30, 2026

1 Syntax

```
(** * Syntax of  $\text{EqCat}_\infty$ 
    Raw terms, dimension, well-formedness, and decidable equality.
    Reference: "Equational Reasoning in  $\infty$ -Categories" §3.1.1, Figure 2 (D1–D4). *)

(* JW: Revision notes (in response to Liron's review).
    Two changes in this file, both prompted by her comments:
    1. The inductive [wf] judgment that mirrors D1–D4 of Figure 2 has
       been MOVED out of this file. It now lives in [Axioms.v] because
       the [wf_comp] case requires the boundary condition
            $\text{src}^{n-k} a == \text{tgt}^{n-k} b$ 
       which is expressed in the theory's own equality [eq_cat], so
       [wf] and [eq_cat] must be defined mutually. The boundary-free
       partial function [dim] stays here unchanged.
    2. Added named aliases [iter_src], [iter_tgt], [iter_id] for
       [Nat.iter k {src,tgt,id}]. Used in the boundary premise of
       [wf_comp] and in IC11–IC13. Pure abbreviations.
    Old [Definition wf := exists n, dim t = Some n] is gone; the new
    inductive [wf] is in [Axioms.v]. *)

Require Import Arith.
Require Import PeanoNat.
Require Import Bool.

(** ** Generators
    The theory is parameterized by a set of generators, each with a fixed
    dimension, and decidable equality. *)
Parameter Generator      : Type.
Parameter gen_dim       : Generator -> nat.
Parameter Generator_eq_dec :
  forall g1 g2 : Generator, {g1 = g2} + {g1 <> g2}.

(** ** Raw terms *)
Inductive Term : Type :=
| gen  : Generator -> Term
| src  : Term -> Term
| tgt  : Term -> Term
| id   : Term -> Term
| comp : nat -> Term -> Term -> Term.

(* JW: NEW. Named aliases for iterated source / target / identity.
```

```

    These appear in IC11–IC13 (originally written with [Nat.iter] inline)
    and in the boundary premise of the new [wf_comp] in [Axioms.v]. *)
Definition iter_src (k : nat) (t : Term) : Term := Nat.iter k src t.
Definition iter_tgt (k : nat) (t : Term) : Term := Nat.iter k tgt t.
Definition iter_id (k : nat) (t : Term) : Term := Nat.iter k id t.

(** ** Dimension
    Partial function on raw terms; [None] marks ill-typed combinations.
    The clauses correspond to D1–D4 of Figure 2:
    D1:  $\dim(\text{src } x) = \dim x - 1$  (when  $\dim x > 0$ )
    D2:  $\dim(\text{tgt } x) = \dim x - 1$  (when  $\dim x > 0$ )
    D3:  $\dim(y \circ [n] x) = d$  (when  $\dim x = \dim y = d > n$ )
    D4:  $\dim(\text{id } x) = \dim x + 1$  *)
Fixpoint dim (t : Term) : option nat :=
  match t with
  | gen g => Some (gen_dim g)
  | src e => match dim e with
    | Some (S n) => Some n
    | _ => None
    end
  | tgt e => match dim e with
    | Some (S n) => Some n
    | _ => None
    end
  | id e => match dim e with
    | Some n => Some (S n)
    | None => None
    end
  | comp k a b =>
    match dim a, dim b with
    | Some da, Some db =>
      if (da =? db) && (k <? da) then Some da else None
    | _, _ => None
    end
  end.

(* JW: WAS:
    Definition wf (t : Term) : Prop := exists n, dim t = Some n.
    Definition wfb (t : Term) : bool :=
      match dim t with Some _ => true | None => false end.

    The old [wf] was a boundary-blind syntactic check. Per Liron's note,
    [wf] should be an inductive judgment AND should check the boundary
    condition for compositions. The proper [wf] now lives in [Axioms.v]
    (mutual with [eq_cat]). The boolean [wfb] is renamed [dimb] below to
    reflect that it tests only the syntactic [dim], not full well-formedness. *)
Definition dimb (t : Term) : bool :=
  match dim t with Some _ => true | None => false end.

(** ** Dimension lemmas (D1–D4 as computational facts) *)
Lemma dim_gen : forall g, dim (gen g) = Some (gen_dim g).
Proof. reflexivity. Qed.
Lemma dim_src : forall e n,
  dim e = Some (S n) -> dim (src e) = Some n.

```

```

Proof. intros e n H; simpl; rewrite H; reflexivity. Qed.
Lemma dim_tgt : forall e n,
  dim e = Some (S n) -> dim (tgt e) = Some n.
Proof. intros e n H; simpl; rewrite H; reflexivity. Qed.
Lemma dim_id : forall e n,
  dim e = Some n -> dim (id e) = Some (S n).
Proof. intros e n H; simpl; rewrite H; reflexivity. Qed.

Lemma dim_comp : forall k a b n,
  dim a = Some n -> dim b = Some n -> k < n ->
  dim (comp k a b) = Some n.
Proof.
  intros k a b n Ha Hb Hk.
  simpl. rewrite Ha, Hb.
  rewrite Nat.eqb_refl; simpl.
  apply Nat.ltb_lt in Hk. rewrite Hk. reflexivity.
Qed.

(** ** Decidable equality on terms *)
Lemma Term_eq_dec : forall t1 t2 : Term, {t1 = t2} + {t1 <> t2}.
Proof.
  decide equality.
  - apply Generator_eq_dec.
  - apply Nat.eq_dec.
Qed.

```

2 Axioms

```

(** * Axioms of EqCat $\infty$ 

The provable-equality relation [t == t'] on raw terms.
Reference: "Equational Reasoning in  $\infty$ -Categories", Figure 2.

Conventions:
- [dim e = Some n] is written explicitly; partial dimensions are
  not implicit.
- For each premise of the form "dim x >= k" in the paper, we
  require [dim e = Some d] with [d >= k].
- Equality of the theory is the inductive relation [eq_cat], NOT
  Coq's built-in [=]. *)

(* JW: Revision notes (in response to Liron's review).
Two changes here, both prompted by her comments:
1. [wf] is now an inductive judgment, defined MUTUALLY with [eq_cat]
   below. The mutual definition is forced by change (2): [wf_comp]
   carries a boundary premise expressed via [eq_cat], and [eq_cat]
   constructors carry [wf] premises, so neither can be defined
   before the other.
2. Composition now checks the required boundary condition. Concretely,
   every constructor that introduces a [comp] (E7, IC5-IC10, IC11,
   IC12, IC13) now carries a [wf (comp ...) d] premise that bundles
   together the dimension constraints AND the boundary equation

```

[iter_src (n-k) a == iter_tgt (n-k) b]. This replaces the loose [dim x = Some d] / [dim y = Some d] / [d > n] premises used previously, which omitted the boundary check entirely.
*Convention reminder: [comp k a b] denotes [a ◦_k b] (natural order; first argument is the left operand of the paper's [◦_k]). *)*

Require Import Arith Lia.

Require Import Syntax.

Reserved Notation "a == b" (at level 70, no associativity).

(JW: NEW. The mutual block defines [wf] alongside [eq_cat] because [wf_comp] uses [eq_cat] for the boundary premise. *)*

Inductive wf : Term -> nat -> **Prop** :=

```
| wf_gen :
  forall g, wf (gen g) (gen_dim g)
| wf_src :
  forall e n, wf e (S n) -> wf (src e) n
| wf_tgt :
  forall e n, wf e (S n) -> wf (tgt e) n
| wf_id :
  forall e n, wf e n -> wf (id e) (S n)
| wf_comp :
  forall k a b n,
    wf a n -> wf b n ->
    k < n ->
    iter_src (n - k) a == iter_tgt (n - k) b ->
    wf (comp k a b) n
```

with eq_cat : Term -> Term -> **Prop** :=

*(** ** Structural equality axioms (E1–E8) *)*

*(** E1: reflexivity. *)*

```
| E1_refl :
  forall x, x == x
```

*(** E2: symmetry. *)*

```
| E2_sym :
  forall x y, x == y -> y == x
```

*(** E3: transitivity. *)*

```
| E3_trans :
  forall x y z, x == y -> y == z -> x == z
```

*(** E4 ("equality preserves dimension") is not a constructor – its conclusion is an equality of [option nat], not of [Term]. It is derivable; see [eq_cat_dim] below. *)*

*(** E5: src is a congruence, where defined. *)*

(JW: WAS:*

```
| E5_src :
  forall x y n,
    x == y -> dim x = Some (S n) -> src x == src y
```

```

    CHANGED: replaced the [dim] premise with [wf] premises on both sides.
    Once dimensions are tracked via the inductive judgment, both [x] and
    [y] are required to be well-formed at the same dimension. *)
| E5_src :
    forall x y n,
      x == y -> wf x (S n) -> wf y (S n) ->
      src x == src y

(** E6: tgt is a congruence, where defined. *)
(* JW: WAS:
    | E6_tgt :
      forall x y n,
        x == y -> dim x = Some (S n) -> tgt x == tgt y
    CHANGED: same pattern as E5. *)
| E6_tgt :
    forall x y n,
      x == y -> wf x (S n) -> wf y (S n) ->
      tgt x == tgt y

(** E7: composition is a congruence. *)
(* JW: WAS:
    | E7_comp :
      forall x y z w n d,
        x == y -> z == w ->
        dim x = Some d -> dim y = Some d ->
        dim z = Some d -> dim w = Some d ->
        d > n ->
        comp n z x == comp n w y
    CHANGED: replaced the four [dim] premises and the [d > n] inequality
    with [wf (comp n x z) d] and [wf (comp n y w) d]. The [wf] premises
    bundle the dimension equalities, the [d > n] constraint, AND the
    required boundary conditions for both compositions in one statement.
    Argument order preserved: [comp n x z] corresponds to [x ◦n z]. *)
| E7_comp :
    forall x y z w n d,
      x == y -> z == w ->
      wf (comp n x z) d -> wf (comp n y w) d ->
      comp n x z == comp n y w

(** E8: id is a congruence. *)
| E8_id :
    forall x y, x == y -> id x == id y

(** ** ∞-categorical axioms (IC1–IC13) *)

(** IC1: dim x ≥ 2 ==> src (src x) == src (tgt x). *)
(* JW: WAS:
    | IC1 :
      forall x n,
        dim x = Some n -> n ≥ 2 ->
        src (src x) == src (tgt x)
    CHANGED: [dim x = Some n] replaced by [wf x n]. Same content, just
    tracked via the new inductive judgment. *)
| IC1 :

```

```

forall x n,
  wf x n -> n >= 2 ->
    src (src x) == src (tgt x)

(** IC2: dim x >= 2 ==> tgt (src x) == tgt (tgt x). *)
(* JW: same change as IC1. *)
| IC2 :
  forall x n,
    wf x n -> n >= 2 ->
      tgt (src x) == tgt (tgt x)

(** IC3: src (id x) == x. *)
(* JW: WAS: [dim x = Some n -> ...]. CHANGED to [wf x n -> ...]. *)
| IC3 :
  forall x n, wf x n -> src (id x) == x

(** IC4: tgt (id x) == x. *)
(* JW: same change as IC3. *)
| IC4 :
  forall x n, wf x n -> tgt (id x) == x

(** IC5: dim x = dim y = n+1 ==> src (y o[n] x) == src x. *)
(* JW: WAS:
  | IC5 :
    forall x y n,
      dim x = Some (S n) -> dim y = Some (S n) ->
        src (comp n y x) == src x
  CHANGED: replaced both [dim] premises with the single
  [wf (comp n y x) (S n)]. This is where Liron's boundary point bites:
  the original axiom permitted forming [comp n y x] without checking
  that [src y == tgt x]. The [wf] premise now folds that check in. *)
| IC5 :
  forall x y n,
    wf (comp n y x) (S n) ->
      src (comp n y x) == src x

(** IC6: dim x = dim y = n+1 ==> tgt (y o[n] x) == tgt y. *)
(* JW: same change as IC5. *)
| IC6 :
  forall x y n,
    wf (comp n y x) (S n) ->
      tgt (comp n y x) == tgt y

(** IC7: dim x = dim y > n+1 ==> src (y o[n] x) == src y o[n] src x. *)
(* JW: WAS:
  | IC7 :
    forall x y n d,
      dim x = Some d -> dim y = Some d -> d > S n ->
        src (comp n y x) == comp n (src y) (src x)
  CHANGED: dimension and boundary on the LHS folded into [wf (comp n y x) d].
  The [d > S n] inequality is kept as a separate premise because it
  is a distinct constraint (the conclusion mentions the lower-dim
  composite [comp n (src y) (src x)] which requires [d - 1 > n]). *)
| IC7 :

```

```

forall x y n d,
  wf (comp n y x) d -> d > S n ->
  src (comp n y x) == comp n (src y) (src x)

(** IC8: dim x = dim y > n+1 ==> tgt (y o[n] x) == tgt y o[n] tgt x. *)
(* JW: same change as IC7. *)
| IC8 :
  forall x y n d,
    wf (comp n y x) d -> d > S n ->
    tgt (comp n y x) == comp n (tgt y) (tgt x)

(** IC9: associativity. dim x = dim y = dim z > n ==>
    (x o[n] y) o[n] z == x o[n] (y o[n] z). *)
(* JW: WAS:
| IC9 :
  forall x y z n d,
    dim x = Some d -> dim y = Some d -> dim z = Some d -> d > n ->
    comp n (comp n x y) z == comp n x (comp n y z)
  CHANGED: replaced the per-variable [dim] premises with [wf] of BOTH
  sides. This is important: the original axiom only constrained the
  variables, but neither nested composition was required to actually
  be well-formed (i.e., have its boundary satisfied). For instance,
  [comp n x y] could be ill-formed even with the dim premises. The
  two [wf] premises now ensure all four boundaries hold. *)
| IC9 :
  forall x y z n d,
    wf (comp n (comp n x y) z) d ->
    wf (comp n x (comp n y z)) d ->
    comp n (comp n x y) z == comp n x (comp n y z)

(** IC10: exchange. dim x = dim y = dim z = dim w > max(m,n), m <> n ==>
    (x o[n] y) o[m] (z o[n] w) == (x o[m] z) o[n] (y o[m] w). *)
(* JW: WAS:
| IC10 :
  forall x y z w m n d,
    dim x = Some d -> dim y = Some d ->
    dim z = Some d -> dim w = Some d ->
    d > m -> d > n -> m <> n ->
    comp m (comp n x y) (comp n z w) ==
    comp n (comp m x z) (comp m y w)
  CHANGED: same pattern as IC9 – [wf] of both sides bundles all
  dimension, level, and boundary constraints (including [m <> n],
  which is implicit in the well-formedness of the nested compositions
  when the boundaries are checked). *)
| IC10 :
  forall x y z w m n d,
    wf (comp m (comp n x y) (comp n z w)) d ->
    wf (comp n (comp m x z) (comp m y w)) d ->
    comp m (comp n x y) (comp n z w) ==
    comp n (comp m x z) (comp m y w)

(** IC11: left identity. d = dim x > n ==>
    x o[n] (id^{d-n} (src^{d-n} x)) == x. *)
(* JW: WAS:

```

```

| IC11 :
  forall x n d,
    dim x = Some d -> d > n ->
      comp n x (Nat.iter (d - n) id (Nat.iter (d - n) src x)) == x
  CHANGED: [Nat.iter (d-n) id (Nat.iter (d-n) src x)] rewritten using
  the new aliases [iter_id (d-n) (iter_src (d-n) x)]. Added a [wf] of
  the LHS so the composition is known to be well-formed (its boundary
  holds automatically by IC3/IC4 iterated, but Coq still needs the
  premise). [wf x d] kept since it's used independently of the LHS. *)
| IC11 :
  forall x n d,
    wf x d -> d > n ->
      wf (comp n x (iter_id (d - n) (iter_src (d - n) x))) d ->
        comp n x (iter_id (d - n) (iter_src (d - n) x)) == x

(** IC12: right identity.  $d = \dim x > n \implies$ 
     $(id^{d-n} (tgt^{d-n} x)) o[n] x = x$ . *)
(* JW: same change as IC11. *)
| IC12 :
  forall x n d,
    wf x d -> d > n ->
      wf (comp n (iter_id (d - n) (iter_tgt (d - n) x)) x) d ->
        comp n (iter_id (d - n) (iter_tgt (d - n) x)) x == x

(** IC13:  $\dim x = \dim y > n, src^{d-n} y = tgt^{d-n} x \implies$ 
     $id (y o[n] x) = id y o[n] id x$ . *)
(* JW: WAS:
| IC13 :
  forall x y n d,
    dim x = Some d -> dim y = Some d -> d > n ->
      Nat.iter (d - n) src y = Nat.iter (d - n) tgt x ->
        id (comp n y x) == comp n (id y) (id x)
  CHANGED: the boundary premise [Nat.iter (d-n) src y = Nat.iter (d-n) tgt x]
  used Coq's built-in [=] but the paper writes it as a Horn premise of
  the theory. Now that [wf (comp n y x) d] expresses the boundary using
  [eq_cat] (which is faithful to the universal Horn presentation), the
  separate equation premise is no longer needed – it is part of [wf]. *)
| IC13 :
  forall x y n d,
    wf (comp n y x) d -> d > n ->
      id (comp n y x) == comp n (id y) (id x)

where "a == b" := (eq_cat a b).

```

3 Metatheory

```

(** * Meta.v
  Metatheory of [wf] and [eq_cat]: well-formedness determines dimension,
  and provable equality preserves dimension.

  Main results:
  - [wf_dim_sound] : wf t d -> dim t = Some d

```



```

- [wf_dim_unique] : wf t d -> wf t d' -> d = d'
- [eq_cat_dim] : x == y -> dim x = dim y
- [eq_cat_dim_agree] : x == y -> wf x d1 -> wf y d2 -> d1 = d2

```

The last item is what [FreeModel.v]'s [eq_cat_dim_agree] re-exports and relies on (it would otherwise have to be [Admitted] there). All results in this file are proved [Qed]. *)

Require Import Arith Lia.

Require Import Syntax **Axioms**.

(** ** wf is sound for the syntactic [dim] *)

Lemma wf_dim_sound : **forall** t d, wf t d -> dim t = Some d.

Proof.

intros t d H. **induction** H.

 - **apply** dim_gen.

 - **apply** dim_src; **exact** IHwf.

 - **apply** dim_tgt; **exact** IHwf.

 - **apply** dim_id; **exact** IHwf.

 - **apply** dim_comp; [**exact** IHwf1 | **exact** IHwf2 | **exact** H1].

Qed.

(** ** wf determines dimension uniquely *)

Lemma wf_dim_unique : **forall** t d d', wf t d -> wf t d' -> d = d'.

Proof.

intros t d d' H1 H2.

apply wf_dim_sound **in** H1. **apply** wf_dim_sound **in** H2.

rewrite H1 **in** H2. **injection** H2; **auto**.

Qed.

(** ** Inversion for compositions *)

Lemma wf_comp_inv : **forall** k a b d,

 wf (comp k a b) d ->

 wf a d /\ wf b d /\ k < d /\ iter_src (d - k) a == iter_tgt (d - k) b.

Proof.

intros k a b d H. **inversion** H; **subst**.

repeat split; **assumption**.

Qed.

(** ** Provable equality preserves dimension.

*Proof by induction on the [eq_cat] derivation. Most cases extract the dimensions of subterms from the [wf] premises via [wf_dim_sound] and then compute; only E8 and transitivity use the induction hypothesis. *)*

Lemma eq_cat_dim : **forall** x y, x == y -> dim x = dim y.

Proof.

intros x y H. **induction** H.

 - (* E1 refl *) **reflexivity**.

 - (* E2 sym *) **symmetry**; **exact** IHeq_cat.

 - (* E3 trans *) **rewrite** IHeq_cat1; **exact** IHeq_cat2.

 - (* E5 src *)

apply wf_dim_sound **in** H0. **apply** wf_dim_sound **in** H1.

simpl. **rewrite** H0, H1. **reflexivity**.

 - (* E6 tgt *)

```

    apply wf_dim_sound in H0. apply wf_dim_sound in H1.
    simpl. rewrite H0, H1. reflexivity.
- (* E7 comp *)
    apply wf_dim_sound in H1. apply wf_dim_sound in H2.
    rewrite H1, H2. reflexivity.
- (* E8 id *)
    simpl. rewrite IHeq_cat. reflexivity.
- (* IC1 : src(src x) == src(tgt x) *)
    apply wf_dim_sound in H. simpl. rewrite H.
    destruct n as [[|n'|]]; try lia. reflexivity.
- (* IC2 *)
    apply wf_dim_sound in H. simpl. rewrite H.
    destruct n as [[|n'|]]; try lia. reflexivity.
- (* IC3 : src(id x) == x *)
    apply wf_dim_sound in H. simpl. rewrite H. reflexivity.
- (* IC4 : tgt(id x) == x *)
    apply wf_dim_sound in H. simpl. rewrite H. reflexivity.
- (* IC5 : src(comp n y x) == src x *)
    apply wf_comp_inv in H. destruct H as (Hy & Hx & Hk & Hb).
    apply wf_dim_sound in Hx. apply wf_dim_sound in Hy.
    simpl. rewrite Hx, Hy.
    rewrite Nat.eqb_refl. simpl.
    assert (Hlt : n <? S n = true) by (apply Nat.ltb_lt; lia).
    rewrite Hlt. reflexivity.
- (* IC6 : tgt(comp n y x) == tgt y *)
    apply wf_comp_inv in H. destruct H as (Hy & Hx & Hk & Hb).
    apply wf_dim_sound in Hx. apply wf_dim_sound in Hy.
    simpl. rewrite Hx, Hy.
    rewrite Nat.eqb_refl. simpl.
    assert (Hlt : n <? S n = true) by (apply Nat.ltb_lt; lia).
    rewrite Hlt. reflexivity.
- (* IC7 : src(comp n y x) == comp n (src y) (src x) *)
    apply wf_comp_inv in H. destruct H as (Hy & Hx & Hk & Hb).
    apply wf_dim_sound in Hx. apply wf_dim_sound in Hy.
    simpl. rewrite Hx, Hy. rewrite Nat.eqb_refl. simpl.
    assert (Hlt : n <? d = true) by (apply Nat.ltb_lt; lia).
    rewrite Hlt.
    destruct d as [|d'|]; [lia]. simpl.
    rewrite Nat.eqb_refl. simpl.
    assert (Hlt2 : n <? d' = true) by (apply Nat.ltb_lt; lia).
    rewrite Hlt2. reflexivity.
- (* IC8 : tgt(comp n y x) == comp n (tgt y) (tgt x) *)
    apply wf_comp_inv in H. destruct H as (Hy & Hx & Hk & Hb).
    apply wf_dim_sound in Hx. apply wf_dim_sound in Hy.
    simpl. rewrite Hx, Hy. rewrite Nat.eqb_refl. simpl.
    assert (Hlt : n <? d = true) by (apply Nat.ltb_lt; lia).
    rewrite Hlt.
    destruct d as [|d'|]; [lia]. simpl.
    rewrite Nat.eqb_refl. simpl.
    assert (Hlt2 : n <? d' = true) by (apply Nat.ltb_lt; lia).
    rewrite Hlt2. reflexivity.
- (* IC9 assoc *)
    apply wf_dim_sound in H. apply wf_dim_sound in H0.
    rewrite H, H0. reflexivity.

```

```

- (* IC10 exchange *)
  apply wf_dim_sound in H. apply wf_dim_sound in H0.
  rewrite H, H0. reflexivity.
- (* IC11 left unit : comp n x (idext) == x *)
  apply wf_dim_sound in H1. apply wf_dim_sound in H.
  rewrite H1, H. reflexivity.
- (* IC12 right unit *)
  apply wf_dim_sound in H1. apply wf_dim_sound in H.
  rewrite H1, H. reflexivity.
- (* IC13 : id(comp n y x) == comp n (id y) (id x) *)
  apply wf_comp_inv in H. destruct H as (Hy & Hx & Hk & Hb).
  apply wf_dim_sound in Hx. apply wf_dim_sound in Hy.
  simpl. rewrite Hx, Hy. rewrite Nat.eqb_refl. simpl.
  assert (Hlt : n <? d = true) by (apply Nat.ltb_lt; lia).
  rewrite Hlt. rewrite Nat.eqb_refl. simpl.
  assert (Hlt2 : n <? S d = true) by (apply Nat.ltb_lt; lia).
  rewrite Hlt2. reflexivity.
Qed.

(** ** The dimension-agreement bridge used by [FreeModel.v]. *)
Lemma eq_cat_dim_agree : forall t1 t2 d1 d2,
  t1 == t2 -> wf t1 d1 -> wf t2 d2 -> d1 = d2.
Proof.
  intros t1 t2 d1 d2 Heq H1 H2.
  apply wf_dim_sound in H1. apply wf_dim_sound in H2.
  apply eq_cat_dim in Heq.
  rewrite H1 in Heq. rewrite H2 in Heq.
  injection Heq; auto.
Qed.

```

4 Normal Forms and the Normalizer

```

(** * NormalForm.v
  Typed normal forms for EqCat $\infty$  and a structurally-recursive normalizer.
  Reference: "Equational Reasoning in  $\infty$ -Categories", §4 (Decidability in
  the free model), Definition 4.1, Lemmas 4.2 and 4.4.

  A normal form is an expression  $id^{\square} end^m a$  with  $end \in \{src, tgt\}$ ,
   $a$  a generator,  $\square, m \geq 0$ , and (when  $m = 0$ ) the two end choices identified.
  We represent these as a record [NF] in CANONICAL form: whenever the
  end-count is 0 we force the end tag to [Es], so that structural equality
  of records is INTENDED to coincide with provable equality of the
  represented terms. The non-trivial direction of that coincidence is
  Lemma 4.2 ([toTerm_inj]), which is established in [Model.v] (via the total
  interpretation [interp]) and re-exported by [Decidable.v]; it is NOT proved
  in this file. Every smart constructor below provably preserves the
  canonical-form invariant. *)

Require Import Arith PeanoNat Lia Bool.
Require Import Syntax Axioms.

Set Implicit Arguments.

```

```

(** ** End operators (src / tgt) *)
Inductive End : Type := Es | Et.

Definition End_eq_dec : forall e1 e2 : End, {e1 = e2} + {e1 <> e2}.
Proof. decide equality. Defined.

Definition end_op (e : End) : Term -> Term :=
  match e with Es => src | Et => tgt end.

Definition iter_end (e : End) (m : nat) (t : Term) : Term :=
  Nat.iter m (end_op e) t.

(** ** Normal forms as records: id^nl end^nm a *)
Record NF : Type := mkNF { nl : nat; ne : End; nm : nat; na : Generator }.

(** Canonical-form invariant: end-count 0 forces end tag [Es]. *)
Definition canonical (u : NF) : Prop := nm u = 0 -> ne u = Es.

(** Interpretation of a normal form as a raw term. *)
Definition toTerm (u : NF) : Term :=
  iter_id (nl u) (iter_end (ne u) (nm u) (gen (na u))).

(** Dimension of a normal form. Well-formed when [nm u <= gen_dim (na u)]. *)
Definition ndim (u : NF) : nat := nl u + (gen_dim (na u) - nm u).
Definition nwf (u : NF) : Prop := nm u <= gen_dim (na u).

(** ** Decidable / boolean equalities *)
Definition Generator_eqb (a b : Generator) : bool :=
  if Generator_eq_dec a b then true else false.
Definition End_eqb (e1 e2 : End) : bool :=
  if End_eq_dec e1 e2 then true else false.

Definition NF_eq_dec : forall u v : NF, {u = v} + {u <> v}.
Proof.
  decide equality.
  - apply Generator_eq_dec.
  - apply Nat.eq_dec.
  - apply End_eq_dec.
  - apply Nat.eq_dec.
Defined.

(** Lemma 4.2 as a boolean test on (canonical) normal forms. *)
Definition NF_eqb (u v : NF) : bool :=
  (nl u =? nl v) && (nm u =? nm v) && Generator_eqb (na u) (na v)
  && (match nm u with 0 => true | _ => End_eqb (ne u) (ne v) end).

(** ** Smart constructors implementing the rewrite rules R1-R11 *)

(** src of a normal form (R6 strips an id; otherwise src wins via R8/def). *)
Definition nf_src (u : NF) : option NF :=
  match nl u with
  | S l' => Some (mkNF l' (ne u) (nm u) (na u))
  | 0    => if nm u <? gen_dim (na u)

```

```

        then Some (mkNF 0 Es (S (nm u)) (na u))
        else None
    end.

(** tgt of a normal form (R7 strips an id; otherwise tgt wins via R9/def). *)
Definition nf_tgt (u : NF) : option NF :=
  match nl u with
  | S l' => Some (mkNF l' (ne u) (nm u) (na u))
  | 0    => if nm u <? gen_dim (na u)
            then Some (mkNF 0 Et (S (nm u)) (na u))
            else None
  end.

(** id of a normal form (always defined). *)
Definition nf_id (u : NF) : NF := mkNF (S (nl u)) (ne u) (nm u) (na u).

(** [src^c] / [tgt^c] of a normal form, as a normal form.
    Stripping an id costs one (IC3/IC4); once the [nl] ids are gone, further
    src/tgt build a fresh src- (resp. tgt-) block (R8/R9: src/tgt "wins"). *)
Definition src_pow (c : nat) (u : NF) : NF :=
  if c <=? nl u then mkNF (nl u - c) (ne u) (nm u) (na u)
  else mkNF 0 Es (nm u + (c - nl u)) (na u).
Definition tgt_pow (c : nat) (u : NF) : NF :=
  if c <=? nl u then mkNF (nl u - c) (ne u) (nm u) (na u)
  else mkNF 0 Et (nm u + (c - nl u)) (na u).

(** [id^c (src^c u)] and [id^c (tgt^c u)] : the right/left identity-extensions
    that appear in the unit laws IC11 / IC12. *)
Definition idext_src (c : nat) (u : NF) : NF :=
  let s := src_pow c u in mkNF (nl s + c) (ne s) (nm s) (na s).
Definition idext_tgt (c : nat) (u : NF) : NF :=
  let s := tgt_pow c u in mkNF (nl s + c) (ne s) (nm s) (na s).

(** Composition of two normal forms at gluing dimension [k] (Lemma 4.4).
    [n] is the common dimension, [c = n - k] the codimension. In the FREE
    model a composite of two normal forms is always degenerate: either the
    right operand is the identity-extension of [src^c] of the left (IC11,
    right unit; result = left), or the left operand is the identity-extension
    of [tgt^c] of the right (IC12, left unit; result = right). *)
Definition nf_comp (k : nat) (u v : NF) : option NF :=
  let n := ndim u in
  if negb (ndim v =? n) then None
  else if negb (k <? n) then None
  else
    let c := n - k in
    if NF_eqb v (idext_src c u) then Some u      (* right unit IC11 *)
    else if NF_eqb u (idext_tgt c v) then Some v  (* left unit IC12 *)
    else None.

(** ** The structural normalizer *)
Fixpoint normalize (t : Term) : option NF :=
  match t with
  | gen a      => Some (mkNF 0 Es 0 a)
  | src e      => match normalize e with Some u => nf_src u | None => None end

```

```

| tgt e      => match normalize e with Some u => nf_tgt u | None => None end
| id e       => match normalize e with Some u => Some (nf_id u) | None => None end
| comp k a b => match normalize a, normalize b with
    | Some u, Some v => nf_comp k u v
    | _, _ => None
    end
end.

(** ** Definitional unfoldings of [normalize] (for clean rewriting). *)
Lemma normalize_gen_eq : forall a, normalize (gen a) = Some (mkNF 0 Es 0 a).
Proof. reflexivity. Qed.
Lemma normalize_src_eq : forall e,
  normalize (src e) = match normalize e with Some u => nf_src u | None => None end.
Proof. reflexivity. Qed.
Lemma normalize_tgt_eq : forall e,
  normalize (tgt e) = match normalize e with Some u => nf_tgt u | None => None end.
Proof. reflexivity. Qed.
Lemma normalize_id_eq : forall e,
  normalize (id e) = match normalize e with Some u => Some (nf_id u) | None => None end.
Proof. reflexivity. Qed.
Lemma normalize_comp_eq : forall k a b,
  normalize (comp k a b)
  = match normalize a, normalize b with
    | Some u, Some v => nf_comp k u v | _, _ => None end.
Proof. reflexivity. Qed.

(** ** Iteration unfolding *)
Lemma iter_end_S : forall e m t,
  iter_end e (S m) t = end_op e (iter_end e m t).
Proof. reflexivity. Qed.

Lemma iter_id_S : forall l t, iter_id (S l) t = id (iter_id l t).
Proof. reflexivity. Qed.

Lemma iter_id_0 : forall t, iter_id 0 t = t.
Proof. reflexivity. Qed.

(** ** Well-formedness of normal forms as raw terms *)
Lemma wf_iter_end : forall e m a, m <= gen_dim a ->
  wf (iter_end e m (gen a)) (gen_dim a - m).
Proof.
  intros e m a Hm. induction m as [|m' IH].
  - rewrite Nat.sub_0_r. apply wf_gen.
  - rewrite iter_end_S.
    assert (Hm' : m' <= gen_dim a) by lia.
    specialize (IH Hm').
    replace (gen_dim a - m') with (S (gen_dim a - S m')) in IH by lia.
    destruct e; simpl end_op.
    + apply (wf_src _ _ IH).
    + apply (wf_tgt _ _ IH).
Qed.

Lemma wf_iter_id : forall l X dx, wf X dx -> wf (iter_id l X) (dx + l).

```

```

Proof.
  intros l X dx H. induction l as [|l' IH].
  - rewrite Nat.add_0_r, iter_id_0. exact H.
  - rewrite iter_id_S. replace (dx + S l') with (S (dx + l')) by lia.
    apply (wf_id _ IH).
Qed.

Lemma toTerm_wf : forall u, nwf u -> wf (toTerm u) (ndim u).
Proof.
  intros u Hu. unfold toTerm, ndim.
  replace (n1 u + (gen_dim (na u) - nm u))
    with ((gen_dim (na u) - nm u) + n1 u) by lia.
  apply wf_iter_id. apply wf_iter_end. exact Hu.
Qed.

(** ** Collapse lemmas: an outer src (resp. tgt) absorbs the end-block.
These realize rewrite rules R8/R9 (driven by IC1/IC2) plus the
definitional [src src = src src] case. *)

Lemma src_tgt_to_src : forall a m, m < gen_dim a ->
  src (iter_end Et m (gen a)) == src (iter_end Es m (gen a)).
Proof.
  intros a m. induction m as [|m' IH]; intros Hm.
  - apply E1_refl.
  - rewrite !iter_end_S; simpl end_op.
    set (T' := iter_end Et m' (gen a)).
    set (S' := iter_end Es m' (gen a)).
    assert (HT' : wf T' (gen_dim a - m')) by (unfold T'; apply wf_iter_end; lia).
    assert (HS' : wf S' (gen_dim a - m')) by (unfold S'; apply wf_iter_end; lia).
    apply (E3_trans (src (tgt T')) (src (src T')) (src (src S'))).
  + apply E2_sym. apply (IC1 T' (gen_dim a - m')); [exact HT' | lia].
  + assert (HsT : wf (src T') (gen_dim a - S m')).
    { apply (wf_src T' (gen_dim a - S m')).
      replace (S (gen_dim a - S m')) with (gen_dim a - m') by lia. exact HT'. }
    assert (HsS : wf (src S') (gen_dim a - S m')).
    { apply (wf_src S' (gen_dim a - S m')).
      replace (S (gen_dim a - S m')) with (gen_dim a - m') by lia. exact HS'. }
    apply (E5_src (src T') (src S') (gen_dim a - S m' - 1)).
    * apply IH; lia.
    * replace (S (gen_dim a - S m' - 1)) with (gen_dim a - S m') by lia. exact HsT.
    * replace (S (gen_dim a - S m' - 1)) with (gen_dim a - S m') by lia. exact HsS.
Qed.

Lemma tgt_src_to_tgt : forall a m, m < gen_dim a ->
  tgt (iter_end Es m (gen a)) == tgt (iter_end Et m (gen a)).
Proof.
  intros a m. induction m as [|m' IH]; intros Hm.
  - apply E1_refl.
  - rewrite !iter_end_S; simpl end_op.
    set (T' := iter_end Et m' (gen a)).
    set (S' := iter_end Es m' (gen a)).
    assert (HT' : wf T' (gen_dim a - m')) by (unfold T'; apply wf_iter_end; lia).
    assert (HS' : wf S' (gen_dim a - m')) by (unfold S'; apply wf_iter_end; lia).
    apply (E3_trans (tgt (src S')) (tgt (tgt S')) (tgt (tgt T'))).

```

```

+ apply (IC2 S' (gen_dim a - m')); [exact HS' | lia].
+ assert (HtT : wf (tgt T') (gen_dim a - S m')).
  { apply (wf_tgt T' (gen_dim a - S m')).
    replace (S (gen_dim a - S m')) with (gen_dim a - m') by lia. exact HT'. }
assert (HtS : wf (tgt S') (gen_dim a - S m')).
{ apply (wf_tgt S' (gen_dim a - S m')).
  replace (S (gen_dim a - S m')) with (gen_dim a - m') by lia. exact HS'. }
apply (E6_tgt (tgt S') (tgt T') (gen_dim a - S m' - 1)).
* apply IH; lia.
* replace (S (gen_dim a - S m' - 1)) with (gen_dim a - S m') by lia. exact HtS.
* replace (S (gen_dim a - S m' - 1)) with (gen_dim a - S m') by lia. exact HtT.

```

Qed.

```

Lemma src_end_collapse : forall a m e, m < gen_dim a ->
  src (iter_end e m (gen a)) == iter_end Es (S m) (gen a).

```

Proof.

```

  intros a m e Hm. rewrite iter_end_S; simpl end_op.
  destruct e.
  - apply E1_refl.
  - apply src_tgt_to_src; exact Hm.

```

Qed.

```

Lemma tgt_end_collapse : forall a m e, m < gen_dim a ->
  tgt (iter_end e m (gen a)) == iter_end Et (S m) (gen a).

```

Proof.

```

  intros a m e Hm. rewrite iter_end_S; simpl end_op.
  destruct e.
  - apply tgt_src_to_tgt; exact Hm.
  - apply E1_refl.

```

Qed.

*(** ** Soundness of the unary smart constructors *)*

```

Lemma nf_id_spec : forall u, nwf u ->
  nwf (nf_id u) /\ ndim (nf_id u) = S (ndim u) /\ toTerm (nf_id u) == id (toTerm u).

```

Proof.

```

  intros u Hu.
  assert (Hn : nwf (nf_id u))
    by (unfold nwf, nf_id; cbn [nl ne nm na]; exact Hu).
  assert (Hd : ndim (nf_id u) = S (ndim u))
    by (unfold ndim, nf_id; cbn [nl ne nm na]; lia).
  assert (Ht : toTerm (nf_id u) == id (toTerm u))
    by (unfold toTerm, nf_id; cbn [nl ne nm na]; rewrite iter_id_S; apply E1_refl).
  repeat split; assumption.

```

Qed.

```

Lemma nf_src_spec : forall u, nwf u -> ndim u > 0 ->
  exists w, nf_src u = Some w /\ nwf w /\ ndim w = ndim u - 1
  /\ toTerm w == src (toTerm u).

```

Proof.

```

  intros u Hu Hd. destruct (nl u) as [l'] eqn:El.
  - (* nl u = 0 : create / extend a src-block (R8 / def) *)
    assert (Hlt : nm u < gen_dim (na u))
      by (unfold ndim in Hd; rewrite El in Hd; cbn in Hd; lia).

```



```

apply Nat.ltb_lt in Hlt as Hltb.
set (w := mkNF 0 Es (S (nm u)) (na u)).
assert (Hs : nf_src u = Some w)
  by (unfold nf_src, w; rewrite El, Hltb; reflexivity).
assert (Hn : nwf w) by (unfold nwf, w; cbn [nl ne nm na]; lia).
assert (Hd2 : ndim w = ndim u - 1)
  by (unfold ndim, w; cbn [nl ne nm na]; rewrite El; cbn; lia).
assert (Ht : toTerm w == src (toTerm u)).
{ unfold toTerm, w; cbn [nl ne nm na]; rewrite El; cbn [iter_id].
  apply E2_sym. apply src_end_collapse; exact Hlt. }
exists w; repeat split; assumption.
- (* nl u = S l' : strip an id (R6 / IC3) *)
set (w := mkNF l' (ne u) (nm u) (na u)).
assert (Hs : nf_src u = Some w)
  by (unfold nf_src, w; rewrite El; reflexivity).
assert (Hn : nwf w) by (unfold nwf, w; cbn [nl ne nm na]; exact Hu).
assert (Hd2 : ndim w = ndim u - 1)
  by (unfold ndim, w; cbn [nl ne nm na]; rewrite El; cbn; lia).
assert (Ht : toTerm w == src (toTerm u)).
{ assert (Htu : toTerm u = id (toTerm w))
  by (unfold toTerm, w; cbn [nl ne nm na]; rewrite El, iter_id_S; reflexivity).
  rewrite Htu. apply E2_sym. apply (IC3 (toTerm w) (ndim w)).
  apply toTerm_wf; exact Hn. }
exists w; repeat split; assumption.
Qed.

Lemma nf_tgt_spec : forall u, nwf u -> ndim u > 0 ->
  exists w, nf_tgt u = Some w /\ nwf w /\ ndim w = ndim u - 1
    /\ toTerm w == tgt (toTerm u).

Proof.
intros u Hu Hd. destruct (nl u) as [l'] eqn:El.
- assert (Hlt : nm u < gen_dim (na u))
  by (unfold ndim in Hd; rewrite El in Hd; cbn in Hd; lia).
apply Nat.ltb_lt in Hlt as Hltb.
set (w := mkNF 0 Et (S (nm u)) (na u)).
assert (Hs : nf_tgt u = Some w)
  by (unfold nf_tgt, w; rewrite El, Hltb; reflexivity).
assert (Hn : nwf w) by (unfold nwf, w; cbn [nl ne nm na]; lia).
assert (Hd2 : ndim w = ndim u - 1)
  by (unfold ndim, w; cbn [nl ne nm na]; rewrite El; cbn; lia).
assert (Ht : toTerm w == tgt (toTerm u)).
{ unfold toTerm, w; cbn [nl ne nm na]; rewrite El; cbn [iter_id].
  apply E2_sym. apply tgt_end_collapse; exact Hlt. }
exists w; repeat split; assumption.
- set (w := mkNF l' (ne u) (nm u) (na u)).
assert (Hs : nf_tgt u = Some w)
  by (unfold nf_tgt, w; rewrite El; reflexivity).
assert (Hn : nwf w) by (unfold nwf, w; cbn [nl ne nm na]; exact Hu).
assert (Hd2 : ndim w = ndim u - 1)
  by (unfold ndim, w; cbn [nl ne nm na]; rewrite El; cbn; lia).
assert (Ht : toTerm w == tgt (toTerm u)).
{ assert (Htu : toTerm u = id (toTerm w))
  by (unfold toTerm, w; cbn [nl ne nm na]; rewrite El, iter_id_S; reflexivity).
  rewrite Htu. apply E2_sym. apply (IC4 (toTerm w) (ndim w)).

```

```

    apply toTerm_wf; exact Hn. }
    exists w; repeat split; assumption.
Qed.

(** ** Retraction:  normalize ◦ toTerm = Some  on canonical, well-formed NFs.

    This is a purely computational correctness check on the normalizer: every
    normal form round-trips. Combined with completeness it yields injectivity
    of [toTerm] (Lemma 4.2). *)

Lemma normalize_end_pos : forall a e m, S m <= gen_dim a ->
  normalize (iter_end e (S m) (gen a)) = Some (mkNF 0 e (S m) a).
Proof.
  intros a e m. revert e. induction m as [|m' IH]; intros e Hm.
  - rewrite iter_end_S. change (iter_end e 0 (gen a)) with (gen a).
    assert (Hb : 0 <? gen_dim a = true) by (apply Nat.ltb_lt; lia).
    destruct e; cbn [end_op].
    + rewrite normalize_src_eq, normalize_gen_eq.
      unfold nf_src; cbn [nl nm na ne]; rewrite Hb; reflexivity.
    + rewrite normalize_tgt_eq, normalize_gen_eq.
      unfold nf_tgt; cbn [nl nm na ne]; rewrite Hb; reflexivity.
  - rewrite iter_end_S.
    assert (Hb : S m' <? gen_dim a = true) by (apply Nat.ltb_lt; lia).
    destruct e; cbn [end_op].
    + rewrite normalize_src_eq, (IH Es ltac:(lia)).
      unfold nf_src; cbn [nl nm na ne]; rewrite Hb; reflexivity.
    + rewrite normalize_tgt_eq, (IH Et ltac:(lia)).
      unfold nf_tgt; cbn [nl nm na ne]; rewrite Hb; reflexivity.
Qed.

Lemma normalize_iter_end : forall a e m, m <= gen_dim a -> (m = 0 -> e = Es) ->
  normalize (iter_end e m (gen a)) = Some (mkNF 0 e m a).
Proof.
  intros a e m Hm Hcanon. destruct m as [|m'].
  - rewrite (Hcanon eq_refl). reflexivity.
  - apply normalize_end_pos; lia.
Qed.

Lemma normalize_iter_id : forall l t w,
  normalize t = Some w ->
  normalize (iter_id l t) = Some (mkNF (nl w + l) (ne w) (nm w) (na w)).
Proof.
  intros l. induction l as [|l' IH]; intros t w Ht.
  - rewrite iter_id_0, Nat.add_0_r, Ht. destruct w; reflexivity.
  - rewrite iter_id_S, normalize_id_eq, (IH t w Ht).
    unfold nf_id; cbn [nl ne nm na].
    replace (nl w + S l') with (S (nl w + l')) by lia. reflexivity.
Qed.

Lemma normalize_toTerm : forall u, nwf u -> canonical u ->
  normalize (toTerm u) = Some u.
Proof.
  intros u Hu Hc. unfold toTerm.
  pose proof (normalize_iter_end Hu Hc) as He.

```

```

rewrite (@normalize_iter_id (nl u) (iter_end (ne u) (nm u) (gen (na u)))
      (mkNF 0 (ne u) (nm u) (na u)) He).
cbn [nl ne nm na]. destruct u; reflexivity.
Qed.

```

5 Free Model (Strict Case)

```

(** * FreeModel.v
    Formalization of the Free Model in the Strict Case for EqCat $\infty$ .
    Reference: "Equational Reasoning in  $\infty$ -Categories"

    In the strict case, the free model is constructed by quotienting
    the well-formed raw terms by the provable equational theory [eq_cat]. *)

Require Import Arith Lia.
Require Import Setoid Morphisms.
Require Import Syntax Axioms Meta.

(** ** 1. Definitional Structure of Free Cells

    A cell in the free model bundles a raw [Term], an explicit dimension [n],
    and a structural proof of the new inductive [wf] predicate at that dimension. *)

Record FreeCell : Type := {
  cell_term : Term;
  cell_dim  : nat;
  cell_proof : wf cell_term cell_dim
}.

(** ** 2. Equivalence Relation on FreeCell

    Two cells are equivalent if their underlying raw terms are provably equal
    under the revised [eq_cat] relation (notation [==]). *)

Definition FreeCell_eq (c1 c2 : FreeCell) : Prop :=
  cell_term c1 == cell_term c2.

Infix "==" := FreeCell_eq (at level 70, no associativity).

(** We establish that [FreeCell_eq] is an equivalence relation using the
    revised structural axioms [E1_refl], [E2_sym], and [E3_trans]. *)

Lemma FreeCell_eq_refl : forall c, c == c.
Proof.
  intros [t d p]; exact (E1_refl t).
Qed.

Lemma FreeCell_eq_sym : forall c1 c2, c1 == c2 -> c2 == c1.
Proof.
  intros [t1 d1 p1] [t2 d2 p2] H; exact (E2_sym t1 t2 H).
Qed.

```

```

Lemma FreeCell_eq_trans : forall c1 c2 c3, c1 == c2 -> c2 == c3 -> c1 == c3.
Proof.
  intros [t1 d1 p1] [t2 d2 p2] [t3 d3 p3] H1 H2; exact (E3_trans t1 t2 t3 H1 H2).
Qed.

(** Register [FreeCell] as a Setoid to unlock rewriting automation within Rocq. *)
Add Parametric Relation : FreeCell FreeCell_eq
  reflexivity proved by FreeCell_eq_refl
  symmetry proved by FreeCell_eq_sym
  transitivity proved by FreeCell_eq_trans
  as FreeCell_setoid.

(** ** 3. Operational Signatures on the Quotient

  We lift [src], [tgt], and [id] to map over [FreeCell].
  For 0-cells, totalization maps the cell to itself to preserve typing constraints. *)

Definition Free_src (c : FreeCell) : FreeCell.
Proof.
  destruct c as [t d p].
  destruct d.
  - (* Totalization boundary for 0-cells: map to itself to preserve type witness 'p' *)
    exact {| cell_term := t; cell_dim := 0; cell_proof := p |}.
  - (* For an (S d) cell, [wf_src] explicitly proves that [src t] is a d-cell *)
    exact {| cell_term := src t; cell_dim := d; cell_proof := wf_src t d p |}.
Defined.

Definition Free_tgt (c : FreeCell) : FreeCell.
Proof.
  destruct c as [t d p].
  destruct d.
  - (* Totalization boundary for 0-cells *)
    exact {| cell_term := t; cell_dim := 0; cell_proof := p |}.
  - (* For an (S d) cell, [wf_tgt] explicitly proves that [tgt t] is a d-cell *)
    exact {| cell_term := tgt t; cell_dim := d; cell_proof := wf_tgt t d p |}.
Defined.

Definition Free_id (c : FreeCell) : FreeCell.
Proof.
  destruct c as [t d p].
  (* Maps an n-cell to an (S n)-cell cleanly using the [wf_id] constructor *)
  exact {| cell_term := id t; cell_dim := S d; cell_proof := wf_id t d p |}.
Defined.

(** ** 4. Metatheoretical Dimension Invariance Lemma

  We assert that provable equality preserves the inductive typing dimensions.
  This acts as our translation bridge for dependent types. *)

Lemma eq_cat_dim_agree : forall t1 t2 d1 d2,
  t1 == t2 -> wf t1 d1 -> wf t2 d2 -> d1 = d2.
Proof.
  exact Meta.eq_cat_dim_agree.
Qed.

```

*(** ** 5. Morphism Adequacy Proofs*

*We declare parametric morphisms to guarantee that the structural operators cleanly respect the Setoid equivalence relation. *)*

Add Parametric **Morphism** : Free_src

with signature FreeCell_eq ==> FreeCell_eq as Free_src_morph.

Proof.

intros [t1 d1 p1] [t2 d2 p2] Heq.

simpl in *.

unfold FreeCell_eq in Heq.

(Step 1: Force dimensions to agree using our metatheory bridge *)*

assert (Hdim: d1 = d2) by exact (eq_cat_dim_agree t1 t2 d1 d2 Heq p1 p2).

subst d2.

(Step 2: Now destructing d1 handles both sides symmetrically *)*

destruct d1.

- *(* Both are 0-cells *)*

exact Heq.

- *(* Both are (S d1) cells; types match perfectly now *)*

exact (E5_src t1 t2 d1 Heq p1 p2).

Qed.

Add Parametric **Morphism** : Free_tgt

with signature FreeCell_eq ==> FreeCell_eq as Free_tgt_morph.

Proof.

intros [t1 d1 p1] [t2 d2 p2] Heq.

simpl in *.

unfold FreeCell_eq in Heq.

assert (Hdim: d1 = d2) by exact (eq_cat_dim_agree t1 t2 d1 d2 Heq p1 p2).

subst d2.

destruct d1.

- exact Heq.

- exact (E6_tgt t1 t2 d1 Heq p1 p2).

Qed.

Add Parametric **Morphism** : Free_id

with signature FreeCell_eq ==> FreeCell_eq as Free_id_morph.

Proof.

intros [t1 d1 p1] [t2 d2 p2] Heq.

simpl in *.

exact (E8_id t1 t2 Heq).

Qed.

*(** ** 6. Composition under the Mutual Architecture*

*To compose [b] and [a] along dimension [k] inside our strict free model, the boundaries must cleanly match. In the revised framework, this proof is embedded directly into the [wf_comp] constructor of the typing relation. *)*

Definition Free_comp (k : nat) (b a : FreeCell)

(Hdim : cell_dim a = cell_dim b)

(Hk : k < cell_dim a)

(Hglue : iter_src (cell_dim a - k) (cell_term a) == iter_tgt (cell_dim a - k)

↪ (cell_term b)) : FreeCell.

```

Proof.
  destruct a as [ta da pa].
  destruct b as [tb db pb].
  simpl in *.
  subst db.
  exact {| cell_term := comp k ta tb;
           cell_dim  := da;
           cell_proof := wf_comp k ta tb da pa pb Hk Hglue |}.
Defined.

```

6 Semantic Model and Injectivity

```

(** * Model.v
  A total semantic interpretation of raw terms into normal-form data,
  used to prove Lemma 4.2 (injectivity of [toTerm], i.e. [toTerm_inj]).

  The carrier is [NF] itself. Operations [m_src], [m_tgt], [m_id], [m_comp]
  are TOTAL (junk on ill-formed inputs). The point: because [interp] is
  total there is no "does it normalize" entanglement, and a single combined
  mutual induction over [eq_cat]/[wf] shows [interp] respects every axiom.
  Distinct canonical normal forms have distinct interpretations, so they are
  not provably equal. *)

Require Import Arith PeanoNat Lia Bool.
Require Import Syntax Axioms Meta NormalForm.

(** ** Total operations on the carrier [NF]. *)

Definition m_src (u : NF) : NF :=
  match nl u with
  | S l => mkNF l (ne u) (nm u) (na u)
  | 0   => mkNF 0 Es (S (nm u)) (na u)
  end.

Definition m_tgt (u : NF) : NF :=
  match nl u with
  | S l => mkNF l (ne u) (nm u) (na u)
  | 0   => mkNF 0 Et (S (nm u)) (na u)
  end.

Definition m_id (u : NF) : NF := mkNF (S (nl u)) (ne u) (nm u) (na u).

(** Total composition, mirroring [nf_comp] but returning junk (the left
    operand) instead of failing. [u] is the left operand, [v] the right. *)
Definition m_comp (k : nat) (u v : NF) : NF :=
  let n := ndim u in
  let c := n - k in
  if NF_eqb v (idext_src c u) then u
  else if NF_eqb u (idext_tgt c v) then v
  else u.

(** ** The interpretation. *)

```

```

Fixpoint interp (t : Term) : NF :=
  match t with
  | gen a      => mkNF 0 Es 0 a
  | src e      => m_src (interp e)
  | tgt e      => m_tgt (interp e)
  | id e       => m_id (interp e)
  | comp k a b => m_comp k (interp a) (interp b)
  end.

(** ** Homomorphism lemmas for the iterated operators. *)
Lemma interp_iter_src : forall c t,
  interp (iter_src c t) = Nat.iter c m_src (interp t).
Proof.
  induction c; intros t; simpl; [reflexivity]].
  unfold iter_src in *. simpl. rewrite IHc. reflexivity.
Qed.

Lemma interp_iter_tgt : forall c t,
  interp (iter_tgt c t) = Nat.iter c m_tgt (interp t).
Proof.
  induction c; intros t; simpl; [reflexivity]].
  unfold iter_tgt in *. simpl. rewrite IHc. reflexivity.
Qed.

Lemma interp_iter_id : forall c t,
  interp (iter_id c t) = Nat.iter c m_id (interp t).
Proof.
  induction c; intros t; simpl; [reflexivity]].
  unfold iter_id in *. simpl. rewrite IHc. reflexivity.
Qed.

(** ** Recovery: interp inverts toTerm on canonical normal forms. *)

Lemma interp_iter_end_Es : forall m a,
  interp (iter_end Es m (gen a)) = mkNF 0 Es m a.
Proof.
  induction m; intros a.
  - reflexivity.
  - rewrite iter_end_S. simpl end_op. simpl interp. rewrite IHm.
    unfold m_src; cbn [n1 ne nm na]. reflexivity.
Qed.

Lemma interp_iter_end_Et : forall m a,
  0 < m -> interp (iter_end Et m (gen a)) = mkNF 0 Et m a.
Proof.
  induction m; intros a Hm; [lia]].
  rewrite iter_end_S. simpl end_op. simpl interp.
  destruct m as [|m'|].
  - simpl. unfold m_tgt; cbn [n1 ne nm na]. reflexivity.
  - rewrite IHm by lia. unfold m_tgt; cbn [n1 ne nm na]. reflexivity.
Qed.

Lemma iter_mid : forall l e m a,
  Nat.iter l m_id (mkNF 0 e m a) = mkNF l e m a.

```

```

Proof.
  induction l; intros e m a; simpl; [reflexivity|].
  rewrite IHL. unfold m_id; cbn [nl ne nm na]. reflexivity.
Qed.

Lemma interp_toTerm : forall u, canonical u -> interp (toTerm u) = u.
Proof.
  intros u Hc. unfold toTerm.
  rewrite interp_iter_id.
  assert (Hend : interp (iter_end (ne u) (nm u) (gen (na u)))
    = mkNF 0 (ne u) (nm u) (na u)).
  { destruct (ne u) eqn:He.
    - rewrite interp_iter_end_Es. reflexivity.
    - destruct (nm u) as [|m'] eqn:Hm.
      + exfalso. unfold canonical in Hc. specialize (Hc Hm).
        rewrite Hc in He. discriminate.
      + rewrite interp_iter_end_Et by lia. reflexivity. }
  rewrite Hend. rewrite iter_mid. destruct u; reflexivity.
Qed.

(** ** Definitional unfoldings of [interp] (controlled rewriting). *)
Lemma interp_comp_eq : forall k a b, interp (comp k a b) = m_comp k (interp a) (interp
  ↦ b).
Proof. reflexivity. Qed.

(** ** Boolean equality facts. *)
Lemma NF_eqb_refl : forall w, NF_eqb w w = true.
Proof.
  intros w. unfold NF_eqb, Generator_eqb, End_eqb.
  rewrite !Nat.eqb_refl.
  destruct (Generator_eq_dec (na w) (na w)); [|congruence].
  destruct (nm w); cbn [andb].
  - reflexivity.
  - destruct (End_eq_dec (ne w) (ne w)); [reflexivity|congruence].
Qed.

(** ** Iterated model operators in closed form. *)
Lemma iter_m_id_eq : forall c u,
  Nat.iter c m_id u = mkNF (nl u + c) (ne u) (nm u) (na u).
Proof.
  induction c; intros u; simpl.
  - rewrite Nat.add_0_r. destruct u; reflexivity.
  - rewrite IHc. unfold m_id; cbn [nl ne nm na]. f_equal. lia.
Qed.

Lemma iter_m_src_pow : forall c u, Nat.iter c m_src u = src_pow c u.
Proof.
  induction c; intros u; simpl.
  - unfold src_pow. cbn. rewrite Nat.sub_0_r. destruct u; reflexivity.
  - rewrite IHc. unfold src_pow. destruct (c <=? nl u) eqn:E1.
    + apply Nat.leb_le in E1. unfold m_src; cbn [nl ne nm na].
      destruct (nl u - c) as [|r] eqn:Er.
      * assert (E2 : (S c <=? nl u) = false) by (apply Nat.leb_gt; lia).
        rewrite E2. f_equal. lia.

```



```

    * assert (E2 : (S c <=? n1 u) = true) by (apply Nat.leb_le; lia).
      rewrite E2. f_equal. lia.
+ apply Nat.leb_gt in E1. unfold m_src; cbn [n1 ne nm na].
  assert (E2 : (S c <=? n1 u) = false) by (apply Nat.leb_gt; lia).
  rewrite E2. f_equal. lia.
Qed.

Lemma iter_m_tgt_pow : forall c u, Nat.iter c m_tgt u = tgt_pow c u.
Proof.
  induction c; intros u; simpl.
- unfold tgt_pow. cbn. rewrite Nat.sub_0_r. destruct u; reflexivity.
- rewrite IHc. unfold tgt_pow. destruct (c <=? n1 u) eqn:E1.
+ apply Nat.leb_le in E1. unfold m_tgt; cbn [n1 ne nm na].
  destruct (n1 u - c) as [|r] eqn:Er.
  * assert (E2 : (S c <=? n1 u) = false) by (apply Nat.leb_gt; lia).
    rewrite E2. f_equal. lia.
  * assert (E2 : (S c <=? n1 u) = true) by (apply Nat.leb_le; lia).
    rewrite E2. f_equal. lia.
+ apply Nat.leb_gt in E1. unfold m_tgt; cbn [n1 ne nm na].
  assert (E2 : (S c <=? n1 u) = false) by (apply Nat.leb_gt; lia).
  rewrite E2. f_equal. lia.
Qed.

(** interp of an identity-extension is exactly the NF-level [idext]. *)
Lemma interp_idext_src : forall c t,
  interp (iter_id c (iter_src c t)) = idext_src c (interp t).
Proof.
  intros c t. rewrite interp_iter_id, interp_iter_src, iter_m_src_pow, iter_m_id_eq.
  unfold idext_src. reflexivity.
Qed.

Lemma interp_idext_tgt : forall c t,
  interp (iter_id c (iter_tgt c t)) = idext_tgt c (interp t).
Proof.
  intros c t. rewrite interp_iter_id, interp_iter_tgt, iter_m_tgt_pow, iter_m_id_eq.
  unfold idext_tgt. reflexivity.
Qed.

(** ** Dimension tracking for the model operators. *)
Lemma ndim_m_id : forall u, ndim (m_id u) = S (ndim u).
Proof. intros u. unfold ndim, m_id; cbn [n1 ne nm na]. lia. Qed.

Lemma ndim_m_src : forall u, ndim u > 0 -> ndim (m_src u) = ndim u - 1.
Proof.
  intros u Hpos. unfold ndim in *. unfold m_src. destruct (n1 u) as [|l].
  - cbn [n1 ne nm na] in *. lia.
  - cbn [n1 ne nm na] in *. lia.
Qed.

Lemma ndim_m_tgt : forall u, ndim u > 0 -> ndim (m_tgt u) = ndim u - 1.
Proof.
  intros u Hpos. unfold ndim in *. unfold m_tgt. destruct (n1 u) as [|l].
  - cbn [n1 ne nm na] in *. lia.
  - cbn [n1 ne nm na] in *. lia.

```

Qed.

Lemma ndim_m_comp : forall k u v m,
 ndim u = m -> ndim v = m -> ndim (m_comp k u v) = m.

Proof.

intros k u v m Hu Hv. unfold m_comp.
 destruct (NF_eqb v (idext_src (ndim u - k) u)); [exact Hu|].
 destruct (NF_eqb u (idext_tgt (ndim u - k) v)); [exact Hv|exact Hu].

Qed.

Lemma interp_ndim : forall t d, wf t d -> ndim (interp t) = d.

Proof.

intros t d H. induction H.
 - unfold ndim; cbn. lia.
 - simpl. rewrite ndim_m_src by lia. lia.
 - simpl. rewrite ndim_m_tgt by lia. lia.
 - simpl. rewrite ndim_m_id. lia.
 - simpl. apply (ndim_m_comp k (interp a) (interp b) n); assumption.

Qed.

*(** ** Closed forms for [src_pow]/[tgt_pow]/[idext_*]. *)*

Lemma src_pow_le : forall c u, c <= nl u ->
 src_pow c u = mkNF (nl u - c) (ne u) (nm u) (na u).

Proof. intros c u H. unfold src_pow. destruct (c <=? nl u) eqn:E;
 [reflexivity | apply Nat.leb_gt in E; lia]. **Qed.**

Lemma src_pow_gt : forall c u, nl u < c ->
 src_pow c u = mkNF 0 Es (nm u + (c - nl u)) (na u).

Proof. intros c u H. unfold src_pow. destruct (c <=? nl u) eqn:E;
 [apply Nat.leb_le in E; lia | reflexivity]. **Qed.**

Lemma tgt_pow_le : forall c u, c <= nl u ->
 tgt_pow c u = mkNF (nl u - c) (ne u) (nm u) (na u).

Proof. intros c u H. unfold tgt_pow. destruct (c <=? nl u) eqn:E;
 [reflexivity | apply Nat.leb_gt in E; lia]. **Qed.**

Lemma tgt_pow_gt : forall c u, nl u < c ->
 tgt_pow c u = mkNF 0 Et (nm u + (c - nl u)) (na u).

Proof. intros c u H. unfold tgt_pow. destruct (c <=? nl u) eqn:E;
 [apply Nat.leb_le in E; lia | reflexivity]. **Qed.**

Lemma idext_src_le : forall c u, c <= nl u -> idext_src c u = u.

Proof.

intros c u H. unfold idext_src. rewrite (src_pow_le c u H). cbn [nl ne nm na].
 replace (nl u - c + c) with (nl u) by lia. destruct u; reflexivity.

Qed.

Lemma idext_src_gt : forall c u, nl u < c ->
 idext_src c u = mkNF c Es (nm u + (c - nl u)) (na u).

Proof.

intros c u H. unfold idext_src. rewrite (src_pow_gt c u H). cbn [nl ne nm na].
 f_equal; lia.

Qed.

```

Lemma idext_tgt_le : forall c u, c <= nl u -> idext_tgt c u = u.
Proof.
  intros c u H. unfold idext_tgt. rewrite (tgt_pow_le c u H). cbn [nl ne nm na].
  replace (nl u - c + c) with (nl u) by lia. destruct u; reflexivity.
Qed.

Lemma idext_tgt_gt : forall c u, nl u < c ->
  idext_tgt c u = mkNF c Et (nm u + (c - nl u)) (na u).
Proof.
  intros c u H. unfold idext_tgt. rewrite (tgt_pow_gt c u H). cbn [nl ne nm na].
  f_equal; lia.
Qed.

(** ** Soundness of [NF_eqb] on canonical normal forms. *)
Lemma NF_eqb_eq : forall u v, canonical u -> canonical v -> NF_eqb u v = true -> u = v.
Proof.
  intros [lu eu mu au] [lv ev mv av] Hcu Hcv H.
  unfold NF_eqb in H; cbn [nl ne nm na] in H.
  apply andb_true_iff in H as [H H4].
  apply andb_true_iff in H as [H H3].
  apply andb_true_iff in H as [H1 H2].
  apply Nat.eqb_eq in H1, H2.
  unfold Generator_eqb in H3. destruct (Generator_eq_dec au av) as [Ha|];
  ↪ [|discriminate].
  subst lv mv av.
  assert (eu = ev) as He.
  { unfold canonical in *; cbn [nm ne] in *.
    destruct mu as [|m'|].
    - rewrite (Hcu eq_refl), (Hcv eq_refl); reflexivity.
    - unfold End_eqb in H4. destruct (End_eq_dec eu ev); [assumption|discriminate]. }
  subst ev. reflexivity.
Qed.

(** ** Lemma 4.4 (model form): a matched boundary forces a unit pattern. *)
Lemma m_comp_boundary : forall c u v,
  src_pow c u = tgt_pow c v -> v = idext_src c u \ / u = idext_tgt c v.
Proof.
  intros c u v H.
  destruct (le_gt_dec c (nl u)) as [Hu|Hu]; destruct (le_gt_dec c (nl v)) as [Hv|Hv].
  - (* both le : u = v *)
    left. rewrite (src_pow_le c u Hu), (tgt_pow_le c v Hv) in H.
    injection H as Hl He Hm Ha. rewrite (idext_src_le c u Hu).
    destruct u as [lu eu mu au], v as [lv ev mv av]; cbn [nl ne nm na] in *.
    subst. f_equal. lia.
  - (* c<=nl u, c>nl v : u = idext_tgt c v *)
    right. rewrite (src_pow_le c u Hu), (tgt_pow_gt c v Hv) in H.
    rewrite (idext_tgt_gt c v Hv).
    destruct u as [lu eu mu au], v as [lv ev mv av]; cbn [nl ne nm na] in *.
    injection H as Hl He Hm Ha. subst. f_equal. lia.
  - (* c>nl u, c<=nl v : v = idext_src c u *)
    left. rewrite (src_pow_gt c u Hu), (tgt_pow_le c v Hv) in H.
    rewrite (idext_src_gt c u Hu).
    destruct u as [lu eu mu au], v as [lv ev mv av]; cbn [nl ne nm na] in *.
    injection H as Hl He Hm Ha. subst. f_equal. lia.

```

```

- (* both gt : impossible, Es = Et *)
  rewrite (src_pow_gt c u Hu), (tgt_pow_gt c v Hv) in H.
  injection H as He Hm Ha. discriminate He.
Qed.

(** ** Canonicity and identities for [idext_*]. *)
Lemma idext_src_canonical : forall c u, canonical u -> canonical (idext_src c u).
Proof.
  intros c u Hc. destruct (le_gt_dec c (nl u)) as [Hle|Hgt].
  - rewrite (idext_src_le c u Hle). exact Hc.
  - rewrite (idext_src_gt c u Hgt). unfold canonical; cbn [nm ne]. intros _; reflexivity.
Qed.

Lemma idext_tgt_canonical : forall c u, canonical u -> canonical (idext_tgt c u).
Proof.
  intros c u Hc. destruct (le_gt_dec c (nl u)) as [Hle|Hgt].
  - rewrite (idext_tgt_le c u Hle). exact Hc.
  - rewrite (idext_tgt_gt c u Hgt). unfold canonical; cbn [nm ne].
    intro H0; exfalso; lia.
Qed.

Lemma idext_tgt_nl : forall c v, nl (idext_tgt c v) = nl (tgt_pow c v) + c.
Proof. intros c v. unfold idext_tgt. cbn [nl]. reflexivity. Qed.

Lemma idext_src_idext_tgt : forall c v, idext_src c (idext_tgt c v) = idext_tgt c v.
Proof. intros c v. apply idext_src_le. rewrite idext_tgt_nl. lia. Qed.

Lemma idext_src_nl : forall c w, nl (idext_src c w) = nl (src_pow c w) + c.
Proof. intros c w. unfold idext_src. cbn [nl]. reflexivity. Qed.

Lemma idext_src_idem : forall c w, idext_src c (idext_src c w) = idext_src c w.
Proof. intros c w. apply idext_src_le. rewrite idext_src_nl. lia. Qed.

Lemma idext_tgt_idem : forall c w, idext_tgt c (idext_tgt c w) = idext_tgt c w.
Proof. intros c w. apply idext_tgt_le. rewrite idext_tgt_nl. lia. Qed.

(** ** Value of [m_comp] under a unit pattern. *)
Lemma m_comp_right : forall k u v,
  v = idext_src (ndim u - k) u -> m_comp k u v = u.
Proof.
  intros k u v Hv. unfold m_comp; cbv zeta. rewrite Hv, NF_eqb_refl. reflexivity.
Qed.

Lemma m_comp_left : forall k u v, canonical u -> canonical v ->
  u = idext_tgt (ndim u - k) v -> m_comp k u v = v.
Proof.
  intros k u v Hcu Hcv Hu. unfold m_comp; cbv zeta. set (c := ndim u - k) in *.
  destruct (NF_eqb v (idext_src c u)) eqn:E1.
  - apply NF_eqb_eq in E1; [| exact Hcv | apply idext_src_canonical; exact Hcu].
    rewrite Hu in E1. rewrite idext_src_idext_tgt in E1. rewrite <- Hu in E1.
    symmetry; exact E1.
  - destruct (NF_eqb u (idext_tgt c v)) eqn:E2.
    + reflexivity.
    + rewrite Hu, NF_eqb_refl in E2. discriminate.

```

Qed.

Lemma m_comp_idsrc : **forall** n d u v,
ndim u = d -> v = idext_src (d - n) u -> m_comp n u v = u.

Proof. **intros** n d u v Hd Hv. **apply** m_comp_right. **rewrite** Hd. **exact** Hv. **Qed.**

Lemma m_comp_idtgt : **forall** n d u v,
ndim u = d -> canonical u -> canonical v ->
u = idext_tgt (d - n) v -> m_comp n u v = v.

Proof. **intros** n d u v Hd Hcu Hcv Hu. **apply** m_comp_left; **try assumption**. **rewrite** Hd. **exact** $\hookrightarrow$ Hu. **Qed.**

*(** Self-composition is degenerate: every branch of [m_comp] returns the left operand, so [m_comp k u u = u] unconditionally. *)*

Lemma m_comp_diag : **forall** k u, m_comp k u u = u.

Proof.

intros k u. **unfold** m_comp; **cbv** zeta.
destruct (NF_eqb u (idext_src (ndim u - k) u)); [**reflexivity**].
destruct (NF_eqb u (idext_tgt (ndim u - k) u)); **reflexivity**.

Qed.

*(** ** [interp] always lands in canonical normal forms. *)*

Lemma m_src_canonical : **forall** w, canonical w -> canonical (m_src w).

Proof.

intros w Hc. **unfold** m_src. **destruct** (nl w).
- **unfold** canonical; **cbn** [nm ne]. **intro** H0; **discriminate**.
- **unfold** canonical **in** *; **cbn** [nm ne] **in** *. **exact** Hc.

Qed.

Lemma m_tgt_canonical : **forall** w, canonical w -> canonical (m_tgt w).

Proof.

intros w Hc. **unfold** m_tgt. **destruct** (nl w).
- **unfold** canonical; **cbn** [nm ne]. **intro** H0; **discriminate**.
- **unfold** canonical **in** *; **cbn** [nm ne] **in** *. **exact** Hc.

Qed.

Lemma m_id_canonical : **forall** w, canonical w -> canonical (m_id w).

Proof. **intros** w Hc. **unfold** m_id, canonical **in** *; **cbn** [nm ne] **in** *. **exact** Hc. **Qed.**

Lemma m_comp_canonical : **forall** k u v,
canonical u -> canonical v -> canonical (m_comp k u v).

Proof.

intros k u v Hu Hv. **unfold** m_comp; **cbv** zeta.
destruct (NF_eqb v (idext_src (ndim u - k) u)); [**exact** Hu].
destruct (NF_eqb u (idext_tgt (ndim u - k) v)); [**exact** Hv|**exact** Hu].

Qed.

Lemma interp_canonical : **forall** t, canonical (interp t).

Proof.

induction t; **simpl**.
- **unfold** canonical; **cbn**. **reflexivity**.
- **apply** m_src_canonical; **exact** IHt.
- **apply** m_tgt_canonical; **exact** IHt.
- **apply** m_id_canonical; **exact** IHt.

```

- apply m_comp_canonical; assumption.
Qed.

(** ** Interaction of [m_src]/[m_tgt] with identity-extensions. *)
Lemma iter_succ_r : forall (f : NF -> NF) c x, Nat.iter (S c) f x = Nat.iter c f (f x).
Proof.
  intros f c x. revert x. induction c; intro x.
  - reflexivity.
  - change (Nat.iter (S (S c)) f x) with (f (Nat.iter (S c) f x)).
    rewrite IHc. reflexivity.
Qed.

Lemma m_src_m_id : forall w, m_src (m_id w) = w.
Proof. intros w. unfold m_src, m_id; cbn [nl ne nm na]. destruct w; reflexivity. Qed.

Lemma m_tgt_m_id : forall w, m_tgt (m_id w) = w.
Proof. intros w. unfold m_tgt, m_id; cbn [nl ne nm na]. destruct w; reflexivity. Qed.

Lemma idext_src_iter : forall c w, idext_src c w = Nat.iter c m_id (Nat.iter c m_src w).
Proof. intros c w. rewrite iter_m_src_pow, iter_m_id_eq. unfold idext_src. reflexivity.
  ↪ Qed.

Lemma idext_tgt_iter : forall c w, idext_tgt c w = Nat.iter c m_id (Nat.iter c m_tgt w).
Proof. intros c w. rewrite iter_m_tgt_pow, iter_m_id_eq. unfold idext_tgt. reflexivity.
  ↪ Qed.

Lemma m_src_idext_src_S : forall c w, m_src (idext_src (S c) w) = idext_src c (m_src w).
Proof.
  intros c w. rewrite !idext_src_iter.
  change (Nat.iter (S c) m_id (Nat.iter (S c) m_src w))
    with (m_id (Nat.iter c m_id (Nat.iter (S c) m_src w))).
  rewrite m_src_m_id. rewrite (iter_succ_r m_src c w). reflexivity.
Qed.

Lemma m_tgt_idext_tgt_S : forall c w, m_tgt (idext_tgt (S c) w) = idext_tgt c (m_tgt w).
Proof.
  intros c w. rewrite !idext_tgt_iter.
  change (Nat.iter (S c) m_id (Nat.iter (S c) m_tgt w))
    with (m_id (Nat.iter c m_id (Nat.iter (S c) m_tgt w))).
  rewrite m_tgt_m_id. rewrite (iter_succ_r m_tgt c w). reflexivity.
Qed.

Lemma idext_src_0 : forall w, idext_src 0 w = w.
Proof. intros w. apply idext_src_le. lia. Qed.

Lemma idext_tgt_0 : forall w, idext_tgt 0 w = w.
Proof. intros w. apply idext_tgt_le. lia. Qed.

Lemma m_id_idext_src : forall c w, m_id (idext_src c w) = idext_src (S c) (m_id w).
Proof.
  intros c w. rewrite !idext_src_iter.
  rewrite (iter_succ_r m_src c (m_id w)), m_src_m_id. reflexivity.
Qed.

```

```

Lemma m_id_idext_tgt : forall c w, m_id (idext_tgt c w) = idext_tgt (S c) (m_id w).
Proof.
  intros c w. rewrite !idext_tgt_iter.
  rewrite (iter_succ_r m_tgt c (m_id w)), m_tgt_m_id. reflexivity.
Qed.

Lemma ndim_m_id_eq : forall u, ndim (m_id u) = S (ndim u).
Proof. exact ndim_m_id. Qed.

(** src/tgt agree once they have to strip an id (model IC1/IC2). *)
Lemma m_tgt_m_src : forall w, m_tgt (m_src w) = m_tgt (m_tgt w).
Proof. intros w. unfold m_src, m_tgt. destruct (nl w); reflexivity. Qed.

Lemma m_src_m_tgt : forall w, m_src (m_tgt w) = m_src (m_src w).
Proof. intros w. unfold m_src, m_tgt. destruct (nl w); reflexivity. Qed.

Lemma m_src_idext_tgt_S : forall c w, m_src (idext_tgt (S c) w) = idext_tgt c (m_tgt w).
Proof.
  intros c w. rewrite !idext_tgt_iter.
  change (Nat.iter (S c) m_id (Nat.iter (S c) m_tgt w))
    with (m_id (Nat.iter c m_id (Nat.iter (S c) m_tgt w))).
  rewrite m_src_m_id, (iter_succ_r m_tgt c w). reflexivity.
Qed.

Lemma m_tgt_idext_src_S : forall c w, m_tgt (idext_src (S c) w) = idext_src c (m_src w).
Proof.
  intros c w. rewrite !idext_src_iter.
  change (Nat.iter (S c) m_id (Nat.iter (S c) m_src w))
    with (m_id (Nat.iter c m_id (Nat.iter (S c) m_src w))).
  rewrite m_tgt_m_id, (iter_succ_r m_src c w). reflexivity.
Qed.

Lemma idext_tgt_collapse : forall j w, idext_tgt (S j) (m_src w) = idext_tgt (S j) (m_tgt
  ↪ w).
Proof.
  intros j w. rewrite !idext_tgt_iter. f_equal.
  rewrite (iter_succ_r m_tgt j (m_src w)), (iter_succ_r m_tgt j (m_tgt w)), m_tgt_m_src.
  reflexivity.
Qed.

Lemma idext_src_collapse : forall j w, idext_src (S j) (m_tgt w) = idext_src (S j) (m_src
  ↪ w).
Proof.
  intros j w. rewrite !idext_src_iter. f_equal.
  rewrite (iter_succ_r m_src j (m_tgt w)), (iter_succ_r m_src j (m_src w)), m_src_m_tgt.
  reflexivity.
Qed.

(** ** Identity-extension tower normalization.
For nested identity-extensions the LARGER codimension wins (absorbs the
smaller). These let an [idext] tower collapse to a single extension. *)

Lemma iter_add : forall (f : NF -> NF) a b x,
  Nat.iter (a + b) f x = Nat.iter a f (Nat.iter b f x).

```

```

Proof. intros f a b x. induction a; simpl; [reflexivity | rewrite IHa; reflexivity]. Qed.

Lemma iter_msrc_mid_cancel : forall c Z, Nat.iter c m_src (Nat.iter c m_id Z) = Z.
Proof.
  induction c; intros Z; [reflexivity]].
  change (Nat.iter (S c) m_id Z) with (m_id (Nat.iter c m_id Z)).
  rewrite (iter_succ_r m_src c (m_id (Nat.iter c m_id Z))), m_src_m_id. apply IHc.
Qed.

Lemma iter_mtgt_mid_cancel : forall c Z, Nat.iter c m_tgt (Nat.iter c m_id Z) = Z.
Proof.
  induction c; intros Z; [reflexivity]].
  change (Nat.iter (S c) m_id Z) with (m_id (Nat.iter c m_id Z)).
  rewrite (iter_succ_r m_tgt c (m_id (Nat.iter c m_id Z))), m_tgt_m_id. apply IHc.
Qed.

Lemma iter_split_src : forall c1 c2 X, c2 <= c1 ->
  Nat.iter c1 m_src X = Nat.iter (c1 - c2) m_src (Nat.iter c2 m_src X).
Proof. intros c1 c2 X H. rewrite <- iter_add. f_equal. lia. Qed.

Lemma iter_split_tgt : forall c1 c2 X, c2 <= c1 ->
  Nat.iter c1 m_tgt X = Nat.iter (c1 - c2) m_tgt (Nat.iter c2 m_tgt X).
Proof. intros c1 c2 X H. rewrite <- iter_add. f_equal. lia. Qed.

Lemma iter_msrc_over_mid : forall c1 c2 W, c2 <= c1 ->
  Nat.iter c1 m_src (Nat.iter c2 m_id W) = Nat.iter (c1 - c2) m_src W.
Proof.
  intros c1 c2 W H.
  rewrite (iter_split_src c1 c2 (Nat.iter c2 m_id W) H), iter_msrc_mid_cancel.
  ↪ reflexivity.
Qed.

Lemma iter_mtgt_over_mid : forall c1 c2 W, c2 <= c1 ->
  Nat.iter c1 m_tgt (Nat.iter c2 m_id W) = Nat.iter (c1 - c2) m_tgt W.
Proof.
  intros c1 c2 W H.
  rewrite (iter_split_tgt c1 c2 (Nat.iter c2 m_id W) H), iter_mtgt_mid_cancel.
  ↪ reflexivity.
Qed.

(** Absorption: outer (smaller-or-equal codim) is swallowed by the inner. *)
Lemma ab_ss : forall c1 c2 X, c1 <= c2 -> idext_src c1 (idext_src c2 X) = idext_src c2 X.
Proof. intros c1 c2 X H. apply idext_src_le. rewrite idext_src_n1. lia. Qed.
Lemma ab_st : forall c1 c2 X, c1 <= c2 -> idext_src c1 (idext_tgt c2 X) = idext_tgt c2 X.
Proof. intros c1 c2 X H. apply idext_src_le. rewrite idext_tgt_n1. lia. Qed.
Lemma ab_ts : forall c1 c2 X, c1 <= c2 -> idext_tgt c1 (idext_src c2 X) = idext_src c2 X.
Proof. intros c1 c2 X H. apply idext_tgt_le. rewrite idext_src_n1. lia. Qed.
Lemma ab_tt : forall c1 c2 X, c1 <= c2 -> idext_tgt c1 (idext_tgt c2 X) = idext_tgt c2 X.
Proof. intros c1 c2 X H. apply idext_tgt_le. rewrite idext_tgt_n1. lia. Qed.

(** Override (outer larger codim, same op): outer wins, inner skipped. *)
Lemma ov_ss : forall c1 c2 X, c2 <= c1 -> idext_src c1 (idext_src c2 X) = idext_src c1 X.
Proof.
  intros c1 c2 X H.

```



```

rewrite (idext_src_iter c1 (idext_src c2 X)), (idext_src_iter c2 X), (idext_src_iter c1
  ↪ X).
f_equal. rewrite iter_msrc_over_mid by lia. rewrite (iter_split_src c1 c2 X H).
  ↪ reflexivity.
Qed.
Lemma ov_tt : forall c1 c2 X, c2 <= c1 -> idext_tgt c1 (idext_tgt c2 X) = idext_tgt c1 X.
Proof.
  intros c1 c2 X H.
  rewrite (idext_tgt_iter c1 (idext_tgt c2 X)), (idext_tgt_iter c2 X), (idext_tgt_iter c1
    ↪ X).
  f_equal. rewrite iter_mtgt_over_mid by lia. rewrite (iter_split_tgt c1 c2 X H).
    ↪ reflexivity.
Qed.

(** Override (outer larger codim, mixed op): outer wins, needs strict <. *)
Lemma ov_st : forall c1 c2 X, c2 < c1 -> idext_src c1 (idext_tgt c2 X) = idext_src c1 X.
Proof.
  intros c1 c2 X H. destruct (le_gt_dec c2 (n1 X)) as [Hle|Hgt].
  - rewrite (idext_tgt_le c2 X Hle). reflexivity.
  - rewrite (idext_tgt_gt c2 X Hgt).
    rewrite (idext_src_gt c1 (mkNF c2 Et (nm X + (c2 - n1 X)) (na X))) by (cbn [n1];
      ↪ lia).
    rewrite (idext_src_gt c1 X) by lia.
    cbn [n1 ne nm na]. f_equal. lia.
Qed.
Lemma ov_ts : forall c1 c2 X, c2 < c1 -> idext_tgt c1 (idext_src c2 X) = idext_tgt c1 X.
Proof.
  intros c1 c2 X H. destruct (le_gt_dec c2 (n1 X)) as [Hle|Hgt].
  - rewrite (idext_src_le c2 X Hle). reflexivity.
  - rewrite (idext_src_gt c2 X Hgt).
    rewrite (idext_tgt_gt c1 (mkNF c2 Es (nm X + (c2 - n1 X)) (na X))) by (cbn [n1];
      ↪ lia).
    rewrite (idext_tgt_gt c1 X) by lia.
    cbn [n1 ne nm na]. f_equal. lia.
Qed.

(** NF discreteness for identity-extensions (for IC10 contradiction branches). *)
Lemma idext_src_codim_inj : forall c1 c2 X,
  n1 X < c1 -> n1 X < c2 -> idext_src c1 X = idext_src c2 X -> c1 = c2.
Proof.
  intros c1 c2 X H1 H2 He.
  rewrite (idext_src_gt c1 X H1), (idext_src_gt c2 X H2) in He.
  injection He; auto.
Qed.

Lemma idext_tgt_codim_inj : forall c1 c2 X,
  n1 X < c1 -> n1 X < c2 -> idext_tgt c1 X = idext_tgt c2 X -> c1 = c2.
Proof.
  intros c1 c2 X H1 H2 He.
  rewrite (idext_tgt_gt c1 X H1), (idext_tgt_gt c2 X H2) in He.
  injection He; auto.
Qed.

Lemma idext_src_tgt_disc : forall c1 c2 X,

```

```

    nl X < c1 -> nl X < c2 -> idext_src c1 X = idext_tgt c2 X -> False.
Proof.
  intros c1 c2 X H1 H2 He.
  rewrite (idext_src_gt c1 X H1), (idext_tgt_gt c2 X H2) in He.
  injection He; intros; discriminate.
Qed.

(** For each identity-extension in the goal, decide its codim against the base's
    nl (guarded so we never re-split the same one), then collapse the small ones. *)
Ltac splitnl :=
  repeat
    (match goal with
    | [ |- context[idext_src ?c ?X] ] =>
      lazymatch goal with
      | [ _ : c <= nl X |- _ ] => fail
      | [ _ : nl X < c |- _ ] => fail
      | _ => destruct (le_gt_dec c (nl X))
      end
    | [ |- context[idext_tgt ?c ?X] ] =>
      lazymatch goal with
      | [ _ : c <= nl X |- _ ] => fail
      | [ _ : nl X < c |- _ ] => fail
      | _ => destruct (le_gt_dec c (nl X))
      end
    end);
  repeat (rewrite idext_src_le by assumption || rewrite idext_tgt_le by assumption).

(** ** Dimension of an identity-extension.
    Whenever the codimension is within the dimension budget [c <= ndim u], an
    identity-extension preserves dimension. (When [c > ndim u] truncation can
    occur, but [c <= ndim u] forces the no-truncation branch.) *)
Lemma ndim_idext_src : forall c u, c <= ndim u -> ndim (idext_src c u) = ndim u.
Proof.
  intros c u H. unfold idext_src, src_pow.
  destruct (c <=? nl u) eqn:E; unfold ndim in *; cbn [nl ne nm na] in *.
  - apply Nat.leb_le in E. lia.
  - apply Nat.leb_gt in E. lia.
Qed.

Lemma ndim_idext_tgt : forall c u, c <= ndim u -> ndim (idext_tgt c u) = ndim u.
Proof.
  intros c u H. unfold idext_tgt, tgt_pow.
  destruct (c <=? nl u) eqn:E; unfold ndim in *; cbn [nl ne nm na] in *.
  - apply Nat.leb_le in E. lia.
  - apply Nat.leb_gt in E. lia.
Qed.

(** A composite whose two unit tests both fail (the "junk" else branch of
    [m_comp]) returns its left operand, provided the only way its left operand
    could be a target-unit of the right is when the two operands coincide.
    This handles the residual case the IC10 exchange leaves behind. *)
Lemma m_comp_junk : forall k u v, canonical u -> canonical v ->
  (u = idext_tgt (ndim u - k) v -> v = u) -> m_comp k u v = u.
Proof.

```

```

intros k u v Hu Hv Himp. unfold m_comp; cbv zeta.
destruct (NF_eqb v (idext_src (ndim u - k) u)) eqn:E1; [reflexivity].
destruct (NF_eqb u (idext_tgt (ndim u - k) v)) eqn:E2; [reflexivity].
apply NF_eqb_eq in E2; [exact Hu | apply idext_tgt_canonical; exact Hv].
apply Himp; exact E2.
Qed.

(** ** The five residual IC10 (exchange) leaves, over an arbitrary canonical
base [B] with [ndim B = d]. Each is one of the boundary-collapse cases left
by the tower-normalization block. *)

Lemma ic10_l1 : forall m n d B, canonical B -> ndim B = d -> d - n > d - m ->
  idext_src (d - m) B = m_comp n (idext_src (d - m) B) (idext_src (d - n) B).
Proof.
  intros m n d B HB Hd Hc. symmetry.
  apply (m_comp_idsrc n d (idext_src (d - m) B) (idext_src (d - n) B)).
  - rewrite ndim_idext_src by (rewrite Hd; lia). exact Hd.
  - symmetry. apply ov_ss. lia.
Qed.

Lemma ic10_l2 : forall m n d B, canonical B -> ndim B = d -> d - n <= d - m ->
  m_comp m (idext_src (d - n) B) (idext_src (d - m) B) = idext_src (d - n) B.
Proof.
  intros m n d B HB Hd Hc.
  apply (m_comp_idsrc m d (idext_src (d - n) B) (idext_src (d - m) B)).
  - rewrite ndim_idext_src by (rewrite Hd; lia). exact Hd.
  - symmetry. apply ov_ss. lia.
Qed.

Lemma ic10_l4 : forall m n d B, canonical B -> ndim B = d -> d - n > d - m ->
  idext_src (d - m) B = m_comp n (idext_tgt (d - n) B) (idext_src (d - m) B).
Proof.
  intros m n d B HB Hd Hc. symmetry.
  apply (m_comp_idtgt n d (idext_tgt (d - n) B) (idext_src (d - m) B)).
  - rewrite ndim_idext_tgt by (rewrite Hd; lia). exact Hd.
  - apply idext_tgt_canonical; exact HB.
  - apply idext_src_canonical; exact HB.
  - symmetry. apply ov_ts. lia.
Qed.

Lemma ic10_l5 : forall m n d B, canonical B -> ndim B = d -> d - n <= d - m ->
  m_comp m (idext_tgt (d - m) (idext_src (d - n) B)) (idext_src (d - n) B)
  = idext_src (d - n) B.
Proof.
  intros m n d B HB Hd Hc.
  assert (Hinner : ndim (idext_src (d - n) B) = d)
    by (rewrite ndim_idext_src by (rewrite Hd; lia); exact Hd).
  apply (m_comp_idtgt m d (idext_tgt (d - m) (idext_src (d - n) B))
    (idext_src (d - n) B)).
  - rewrite ndim_idext_tgt by (rewrite Hinner; lia). exact Hinner.
  - apply idext_tgt_canonical, idext_src_canonical; exact HB.
  - apply idext_src_canonical; exact HB.
  - reflexivity.
Qed.

```

```

Lemma ic10_l3 : forall m n d B, canonical B -> ndim B = d -> d - n > d - m ->
  m_comp m (idext_src (d - m) B) (idext_tgt (d - n) B)
  = m_comp n (idext_tgt (d - n) B) (idext_src (d - m) B).
Proof.
  intros m n d B HB Hd Hc.
  assert (HR : m_comp n (idext_tgt (d - n) B) (idext_src (d - m) B)
    = idext_src (d - m) B).
  { apply (m_comp_idtgt n d).
    - rewrite ndim_idext_tgt by (rewrite Hd; lia). exact Hd.
    - apply idext_tgt_canonical; exact HB.
    - apply idext_src_canonical; exact HB.
    - symmetry. apply ov_ts. lia. }
  rewrite HR.
  apply m_comp_junk.
  - apply idext_src_canonical; exact HB.
  - apply idext_tgt_canonical; exact HB.
  - intro H.
    rewrite ndim_idext_src in H by (rewrite Hd; lia).
    rewrite Hd in H. rewrite ab_tt in H by lia.
    symmetry; exact H.
Qed.

(** ** Combined mutual induction principle for [eq_cat] and [wf]. *)
Scheme eq_cat_min := Minimality for eq_cat Sort Prop
  with wf_min := Minimality for wf Sort Prop.
Combined Scheme eq_cat_wf_mut from eq_cat_min, wf_min.

(** The wf-carried invariant: model-level boundary for compositions. *)
(** Recursive model well-formedness: every nested composition's boundary
  matches at the model level. This is what lets the IC9/IC10 cases reach
  the boundaries of the *inner* composites. *)
Fixpoint Qwf (t : Term) (d : nat) : Prop :=
  match t with
  | gen _      => True
  | src e      => Qwf e (S d)
  | tgt e      => Qwf e (S d)
  | id e       => match d with 0 => True | S n => Qwf e n end
  | comp k a b => src_pow (d - k) (interp a) = tgt_pow (d - k) (interp b)
    /\ Qwf a d /\ Qwf b d
  end.

(** The top boundary in [src_pow]/[tgt_pow] form. *)
Lemma Qwf_boundary : forall k a b d, Qwf (comp k a b) d ->
  src_pow (d - k) (interp a) = tgt_pow (d - k) (interp b).
Proof. intros k a b d H. exact (proj1 H). Qed.

(** Resolve a composite whose boundary's unit pattern is a hypothesis, in all
  hypotheses and the goal. Used to flatten the IC10 (exchange) case tree. *)
Ltac mres :=
  repeat
    ((rewrite m_comp_idsrc in * by eassumption)
    || (rewrite m_comp_idtgt in * by (eassumption || apply interp_canonical))).

```

```

(** Resolve the one composite pinned by boundary equation [E], in all hyps + goal. *)
Ltac mres1 E :=
  try first
    [ erewrite (m_comp_idsrc _ _ _ _ E) in * by eassumption
    | erewrite (m_comp_idtgt _ _ _ _ E) in * by (eauto using interp_canonical) ].

(** Resolve the composite pinned by [E] inside hypothesis [H]. *)
Ltac mresH H E :=
  try first
    [ erewrite (m_comp_idsrc _ _ _ _ E) in H by eassumption
    | erewrite (m_comp_idtgt _ _ _ _ E) in H by (eauto using interp_canonical) ].

(** Repeatedly resolve any composite pinned by a boundary equation in context. *)
Ltac mstep :=
  match goal with
  | [ E : ?v = idext_src ?c ?u |- _ ] =>
    erewrite (m_comp_idsrc _ _ u v _ E) in * by eassumption
  | [ E : ?u = idext_tgt ?c ?v |- _ ] =>
    erewrite (m_comp_idtgt _ _ u v _ _ E) in *
    by (first [ eassumption | apply interp_canonical ])
  end.
Ltac mresall := repeat mstep.

(** Goal-directed: resolve a composite in the goal whose operands have a
    matching boundary equation in context. *)
Ltac gres :=
  repeat
    match goal with
    | [ E : ?v = idext_src ?c ?u |- context[m_comp ?k ?u ?v] ] =>
      rewrite (m_comp_idsrc k _ u v ltac:(eassumption) E)
    | [ E : ?u = idext_tgt ?c ?v |- context[m_comp ?k ?u ?v] ] =>
      rewrite (m_comp_idtgt k _ u v ltac:(eassumption)
        (interp_canonical _) (interp_canonical _) E)
    end.

(** ** Model soundness: [interp] respects provable equality.
    Proved by the combined mutual induction; the [comp] axiom cases (IC5–IC13)
    are the substantive content. The IC10 (exchange) case reduces, after
    tower-normalization, to the five boundary-collapse leaves discharged by
    [ic10_l1]–[ic10_l5] above. *)
Theorem interp_sound_mut :
  (forall x y, x == y -> interp x = interp y)
  /\ (forall t d, wf t d -> Qwf t d).
Proof.
  apply eq_cat_wf_mut.
  - (* E1 refl *) intros x. reflexivity.
  - (* E2 sym *) intros x y _ IH. symmetry; exact IH.
  - (* E3 trans *) intros x y z _ IH1 _ IH2. rewrite IH1; exact IH2.
  - (* E5 src *) intros x y n _ IH _ . simpl. rewrite IH. reflexivity.
  - (* E6 tgt *) intros x y n _ IH _ . simpl. rewrite IH. reflexivity.
  - (* E7 comp *) intros x y z w n d _ IHxy _ IHzw _ . simpl. rewrite IHxy, IHzw.
    ↪ reflexivity.
  - (* E8 id *) intros x y _ IH. simpl. rewrite IH. reflexivity.
  - (* IC1 *) intros x n _ .

```

```

    simpl. unfold m_src, m_tgt. destruct (nl (interp x)) as [|[l]]; reflexivity.
- (* IC2 *) intros x n _ .
    simpl. unfold m_src, m_tgt. destruct (nl (interp x)) as [|[l]]; reflexivity.
- (* IC3 src(id x) = x *) intros x n _ .
    simpl. unfold m_src, m_id; cbn [nl ne nm na]. destruct (interp x); reflexivity.
- (* IC4 tgt(id x) = x *) intros x n _ .
    simpl. unfold m_tgt, m_id; cbn [nl ne nm na]. destruct (interp x); reflexivity.
- (* IC5 src(comp) = src x *) intros x y n Hwf IHwf.
    apply wf_comp_inv in Hwf. destruct Hwf as (Hy & Hx & Hk & Hbnd).
    apply Qwf_boundary in IHwf. replace (S n - n) with 1 in IHwf by lia.
    pose proof (interp_ndim _ _ Hy) as Hny.
    cbn [interp].
    destruct (m_comp_boundary 1 (interp y) (interp x) IHwf) as [HA | HB].
    + rewrite (m_comp_right n (interp y) (interp x)).
      2:{ rewrite Hny. replace (S n - n) with 1 by lia. exact HA. }
      rewrite HA, m_src_idext_src_S, idext_src_0. reflexivity.
    + rewrite (m_comp_left n (interp y) (interp x)).
      * reflexivity.
      * apply interp_canonical.
      * apply interp_canonical.
      * rewrite Hny. replace (S n - n) with 1 by lia. exact HB.
- (* IC6 tgt(comp) = tgt y *) intros x y n Hwf IHwf.
    apply wf_comp_inv in Hwf. destruct Hwf as (Hy & Hx & Hk & Hbnd).
    apply Qwf_boundary in IHwf. replace (S n - n) with 1 in IHwf by lia.
    pose proof (interp_ndim _ _ Hy) as Hny.
    cbn [interp].
    destruct (m_comp_boundary 1 (interp y) (interp x) IHwf) as [HA | HB].
    + rewrite (m_comp_right n (interp y) (interp x)).
      * reflexivity.
      * rewrite Hny. replace (S n - n) with 1 by lia. exact HA.
    + rewrite (m_comp_left n (interp y) (interp x)).
      2: apply interp_canonical.
      2: apply interp_canonical.
      2:{ rewrite Hny. replace (S n - n) with 1 by lia. exact HB. }
      rewrite HB, m_tgt_idext_tgt_S, idext_tgt_0. reflexivity.
- (* IC7 src(comp) = comp(src)(src) *) intros x y n d Hwf IHwf Hd.
    apply wf_comp_inv in Hwf. destruct Hwf as (Hy & Hx & Hk & Hbnd).
    apply Qwf_boundary in IHwf.
    pose proof (interp_ndim _ _ Hy) as Hny.
    cbn [interp].
    destruct (d - n) as [|[c']] eqn:Hc; try lia.
    destruct (m_comp_boundary (S (S c')) (interp y) (interp x) IHwf) as [HA | HB].
    + rewrite (m_comp_right n (interp y) (interp x)).
      2:{ rewrite Hny, Hc. exact HA. }
      rewrite HA, m_src_idext_src_S.
      rewrite (m_comp_right n (m_src (interp y)) (idext_src (S c') (m_src (interp y)))).
      2:{ rewrite ndim_m_src by (rewrite Hny; lia). rewrite Hny.
        replace (d - 1 - n) with (S c') by lia. reflexivity. }
      reflexivity.
    + rewrite (m_comp_left n (interp y) (interp x)).
      2: apply interp_canonical.
      2: apply interp_canonical.
      2:{ rewrite Hny, Hc. exact HB. }
      rewrite (m_comp_left n (m_src (interp y)) (m_src (interp x))).

```

```

* reflexivity.
* apply m_src_canonical; apply interp_canonical.
* apply m_src_canonical; apply interp_canonical.
* rewrite ndim_m_src by (rewrite Hny; lia). rewrite Hny.
  replace (d - 1 - n) with (S c'') by lia.
  rewrite HB, m_src_idext_tgt_S. symmetry; apply idext_tgt_collapse.
- (* IC8 tgt(comp) = comp(tgt)(tgt) *) intros x y n d Hwf IHwf Hd.
  apply wf_comp_inv in Hwf. destruct Hwf as (Hy & Hx & Hk & Hbnd).
  apply Qwf_boundary in IHwf.
  pose proof (interp_ndim _ _ Hy) as Hny.
  cbn [interp].
  destruct (d - n) as [||c''] eqn:Hc; try lia.
  destruct (m_comp_boundary (S (S c'')) (interp y) (interp x) IHwf) as [HA | HB].
+ rewrite (m_comp_right n (interp y) (interp x)).
  2:{ rewrite Hny, Hc. exact HA. }
  rewrite (m_comp_right n (m_tgt (interp y)) (m_tgt (interp x))).
* reflexivity.
* rewrite ndim_m_tgt by (rewrite Hny; lia). rewrite Hny.
  replace (d - 1 - n) with (S c'') by lia.
  rewrite HA, m_tgt_idext_src_S. symmetry; apply idext_src_collapse.
+ rewrite (m_comp_left n (interp y) (interp x)).
  2: apply interp_canonical.
  2: apply interp_canonical.
  2:{ rewrite Hny, Hc. exact HB. }
  rewrite (m_comp_left n (m_tgt (interp y)) (m_tgt (interp x))).
* reflexivity.
* apply m_tgt_canonical; apply interp_canonical.
* apply m_tgt_canonical; apply interp_canonical.
* rewrite ndim_m_tgt by (rewrite Hny; lia). rewrite Hny.
  replace (d - 1 - n) with (S c'') by lia.
  rewrite HB, m_tgt_idext_tgt_S. reflexivity.
- (* IC9 assoc *) intros x y z n d Hwf1 IH1 Hwf2 IH2.
  apply wf_comp_inv in Hwf1. destruct Hwf1 as (Hxy & Hz & Hk & _).
  apply wf_comp_inv in Hxy. destruct Hxy as (Hx & Hy & _ & _).
  pose proof (interp_ndim _ _ Hx) as HnX.
  pose proof (interp_ndim _ _ Hy) as HnY.
  destruct IH1 as (OL & IH1xy & QZ). destruct IH1xy as (IL & QX & QY).
  destruct IH2 as (OR & QX2 & IH2yz). destruct IH2yz as (IR & QY2 & QZ2).
  cbn [interp].
  destruct (m_comp_boundary (d - n) (interp x) (interp y) IL) as [ILa | ILb];
  destruct (m_comp_boundary (d - n) (interp y) (interp z) IR) as [IRa | IRb].
+ rewrite (m_comp_idsrc n d (interp x) (interp y) HnX ILa).
  rewrite (m_comp_idsrc n d (interp y) (interp z) HnY IRa).
  assert (HZ : interp z = interp y) by (rewrite IRa, ILa, idext_src_idem;
    ↪ reflexivity).
  rewrite HZ. reflexivity.
+ rewrite (m_comp_idsrc n d (interp x) (interp y) HnX ILa).
  rewrite (m_comp_idtgt n d (interp y) (interp z) HnY
    (interp_canonical _) (interp_canonical _) IRb).
  reflexivity.
+ rewrite (m_comp_idtgt n d (interp x) (interp y) HnX
  (interp_canonical _) (interp_canonical _) ILb).
  rewrite (m_comp_idsrc n d (interp y) (interp z) HnY IRa).
  rewrite (m_comp_idtgt n d (interp x) (interp y) HnX

```

```

      (interp_canonical _) (interp_canonical _) ILb).
    reflexivity.
+ rewrite (m_comp_idtgt n d (interp x) (interp y) HnX
      (interp_canonical _) (interp_canonical _) ILb).
    rewrite (m_comp_idtgt n d (interp y) (interp z) HnY
      (interp_canonical _) (interp_canonical _) IRb).
    assert (HX : interp x = idext_tgt (d - n) (interp z))
      by (rewrite ILb, IRb, idext_tgt_idem; reflexivity).
    rewrite (m_comp_idtgt n d (interp x) (interp z) HnX
      (interp_canonical _) (interp_canonical _) HX).
    reflexivity.
- (* IC10 exchange *) intros x y z w m n d Hwf1 IH1 Hwf2 IH2.
  apply wf_comp_inv in Hwf1. destruct Hwf1 as (Hxy & Hzw & Hm & _).
  apply wf_comp_inv in Hxy. destruct Hxy as (Hx & Hy & _ & _).
  apply wf_comp_inv in Hzw. destruct Hzw as (Hz & Hw & _ & _).
  pose proof (interp_ndim _ _ Hx) as HnX. pose proof (interp_ndim _ _ Hy) as HnY.
  pose proof (interp_ndim _ _ Hz) as HnZ. pose proof (interp_ndim _ _ Hw) as HnW.
  pose proof (interp_canonical x) as Cx. pose proof (interp_canonical y) as Cy.
  pose proof (interp_canonical z) as Cz. pose proof (interp_canonical w) as Cw.
  destruct IH1 as (OL & (Bxy & _ & _) & (Bzw & _ & _)).
  destruct IH2 as (OR & (Bxz & _ & _) & (Byw & _ & _)).
  cbn [interp] in OL, OR |- *.
  destruct (m_comp_boundary (d - n) _ _ Bxy) as [Exy | Exy];
  destruct (m_comp_boundary (d - n) _ _ Bzw) as [Ezw | Ezw];
  destruct (m_comp_boundary (d - m) _ _ Bxz) as [Exz | Exz];
  destruct (m_comp_boundary (d - m) _ _ Byw) as [Eyw | Eyw];
  mresH OL Exy; mresH OL Ezw; mresH OR Exz; mresH OR Eyw;
  destruct (m_comp_boundary (d - m) _ _ OL) as [EOL | EOL];
  destruct (m_comp_boundary (d - n) _ _ OR) as [EOR | EOR];
  gres.
  (* Residuals: equalities between iterated identity-extension towers.
     Express everything over a common base, then normalize towers (larger
     codim wins) after fixing the codim order. *)
  all: try reflexivity.
  all: try (destruct (le_gt_dec (d - n) (d - m));
    repeat match goal with
      | [ H : _ = idext_src _ _ |- _ ] => rewrite H in *; clear H
      | [ H : _ = idext_tgt _ _ |- _ ] => rewrite H in *; clear H
    end;
    repeat (first
      [ rewrite ab_ss by lia | rewrite ab_st by lia
        | rewrite ab_ts by lia | rewrite ab_tt by lia
        | rewrite ov_ss by lia | rewrite ov_tt by lia
        | rewrite ov_st by lia | rewrite ov_ts by lia
        | rewrite m_comp_diag ];
      try reflexivity)).
  (* Tower normalization + m_comp_diag close 59 of 64 leaves. The last 5 are
     codim-mismatch goals that hold when codims <= nl X (extensions collapse)
     and otherwise mark a contradiction branch. A guarded codim-vs-nl split
     (splitnl) collapses the small ones but the contradiction branches need a
     config-specific argument; left as the final gap. *)
  (* The 5 residual leaves are boundary-collapse cases; each matches exactly
     one of the [ic10_l1..l5] lemmas over the relevant canonical base
     ([interp x], [interp y] or [interp z]). The side-conditions ([canonical],

```



```

    [ndim _ = d], codim order) are discharged uniformly; [eapply] shelves the
    [ndim] obligations, so we also solve them after [Unshelve]. *)
all: first
  [ eapply ic10_l1 | eapply ic10_l2 | eapply ic10_l3
    | eapply ic10_l4 | eapply ic10_l5 ].
all: try (apply interp_canonical).
all: try (apply interp_ndim; assumption).
all: try lia.
Unshelve.
all: try (apply interp_canonical).
all: try (apply interp_ndim; assumption).
all: try lia.
- (* IC11 left unit *) intros x n d Hx IHx Hd Hwf IHwf.
  rewrite interp_comp_eq. rewrite interp_idext_src.
  unfold m_comp. rewrite (interp_ndim x d Hx).
  rewrite NF_eqb_refl. reflexivity.
- (* IC12 right unit *) intros x n d Hx IHx Hd Hwf IHwf.
  rewrite interp_comp_eq, interp_idext_tgt.
  apply wf_comp_inv in Hwf. destruct Hwf as (HL & Hxr & HK & Hbnd).
  apply interp_ndim in HL. rewrite interp_idext_tgt in HL.
  apply m_comp_left.
+ apply idext_tgt_canonical. apply interp_canonical.
+ apply interp_canonical.
+ rewrite HL. reflexivity.
- (* IC13 id(comp) = comp(id)(id) *) intros x y n d Hwf IHwf Hd.
  apply wf_comp_inv in Hwf. destruct Hwf as (Hy & Hx & HK & Hbnd).
  apply Qwf_boundary in IHwf.
  pose proof (interp_ndim _ _ Hy) as Hny.
  cbn [interp]. remember (d - n) as c eqn:Hc.
  destruct (m_comp_boundary c (interp y) (interp x) IHwf) as [HA | HB].
+ rewrite (m_comp_right n (interp y) (interp x)).
  2:{ rewrite Hny, <- Hc. exact HA. }
  rewrite (m_comp_right n (m_id (interp y)) (m_id (interp x))).
  2:{ rewrite ndim_m_id, Hny. replace (S d - n) with (S c) by lia.
    rewrite HA, m_id_idext_src. reflexivity. }
  reflexivity.
+ rewrite (m_comp_left n (interp y) (interp x)).
  2: apply interp_canonical.
  2: apply interp_canonical.
  2:{ rewrite Hny, <- Hc. exact HB. }
  rewrite (m_comp_left n (m_id (interp y)) (m_id (interp x))).
  2: apply m_id_canonical; apply interp_canonical.
  2: apply m_id_canonical; apply interp_canonical.
  2:{ rewrite ndim_m_id, Hny. replace (S d - n) with (S c) by lia.
    rewrite HB, m_id_idext_tgt. reflexivity. }
  reflexivity.
- (* wf_gen *) intros g. exact I.
- (* wf_src *) intros e n He QHe. exact QHe.
- (* wf_tgt *) intros e n He QHe. exact QHe.
- (* wf_id *) intros e n He QHe. exact QHe.
- (* wf_comp *) intros k a b n Ha QHa Hb QHb Hk Hbnd Pbnd.
  simpl. split.
+ rewrite interp_iter_src, iter_m_src_pow in Pbnd.
  rewrite interp_iter_tgt, iter_m_tgt_pow in Pbnd. exact Pbnd.

```

```

+ split; [exact QHa | exact QHb].
Qed.

(** ** The injectivity lemma (Lemma 4.2). *)
Corollary interp_sound : forall x y, x == y -> interp x = interp y.
Proof. apply interp_sound_mut. Qed.

Theorem toTerm_inj : forall u v,
  nwf u -> canonical u -> nwf v -> canonical v ->
  toTerm u == toTerm v -> u = v.
Proof.
  intros u v _ Hcu _ Hcv H.
  apply interp_sound in H.
  rewrite (interp_toTerm u Hcu) in H.
  rewrite (interp_toTerm v Hcv) in H.
  exact H.
Qed.

```

7 Decidability

```

(** * Decidable.v
    Decidability of provable equality in the strict free model (Theorem 4.6).

    Architecture (see header discussion in the development):
    - [normalize_sound]      : normalize is sound w.r.t. ==                [ADMITTED]
    - [toTerm_inj]          : Lemma 4.2 – canonical normal forms are
                              not == unless equal (the deep lemma;
                              confluence / semantic core)                [ADMITTED]
    - [normalize_total]     : normalize succeeds on wf terms (uses injectivity
                              in the comp case)
    - [normalize_complete]  : == implies equal normal forms (from the above)
    - [free_cell_decidable]: Theorem 4.6 (from completeness + soundness + totality)

    PROOF STATUS. This file is admit-free. [toTerm_inj] is discharged by
    [Model.toTerm_inj] (proved in [Model.v] via the combined [eq_cat]/[wf]
    mutual induction [Model.interp_sound_mut]); [normalize_sound] is a
    corollary of the strengthened [normalize_total]; and [src_pow_sound],
    [tgt_pow_sound], [nf_comp_glue] are proved below. Hence
    [Print Assumptions free_cell_decidable] lists only the three intended
    abstract generator parameters ([Generator], [gen_dim], [Generator_eq_dec]). *)

Require Import Arith PeanoNat Lia Bool.
Require Import Syntax Axioms Meta NormalForm FreeModel Model.

(* Set Implicit Arguments. *)

(** ** Iteration / arithmetic helpers *)

Lemma iter_id_add : forall a b X, iter_id (a + b) X = iter_id a (iter_id b X).
Proof. induction a; intros; simpl; [reflexivity | rewrite IHa; reflexivity]. Qed.

Lemma iter_src_add : forall a b X, iter_src (a + b) X = iter_src a (iter_src b X).

```

```

Proof. induction a; intros; simpl; [reflexivity | rewrite IHa; reflexivity]. Qed.

Lemma iter_tgt_add : forall a b X, iter_tgt (a + b) X = iter_tgt a (iter_tgt b X).
Proof. induction a; intros; simpl; [reflexivity | rewrite IHa; reflexivity]. Qed.

(** ** Canonical-form preservation by the unary smart constructors *)

Lemma nf_src_canonical : forall u w, canonical u -> nf_src u = Some w -> canonical w.
Proof.
  intros u w Hc Hs. unfold nf_src in Hs. destruct (nl u) eqn:E.
  - destruct (nm u <? gen_dim (na u)) eqn:Eb; [|discriminate].
    injection Hs; intro Hw; subst w.
    unfold canonical; cbn [nm ne]. intro H0; discriminate H0.
  - injection Hs; intro Hw; subst w.
    unfold canonical in *; cbn [nm ne] in *. exact Hc.
Qed.

Lemma nf_tgt_canonical : forall u w, canonical u -> nf_tgt u = Some w -> canonical w.
Proof.
  intros u w Hc Hs. unfold nf_tgt in Hs. destruct (nl u) eqn:E.
  - destruct (nm u <? gen_dim (na u)) eqn:Eb; [|discriminate].
    injection Hs; intro Hw; subst w.
    unfold canonical; cbn [nm ne]. intro H0; discriminate H0.
  - injection Hs; intro Hw; subst w.
    unfold canonical in *; cbn [nm ne] in *. exact Hc.
Qed.

(** ** Commutation lemmas relating normalize-level [src_pow]/[tgt_pow]/[idext]
    to the term-level iterated operators. NF-only, no circularity. *)

(** [src_pow_sound] and [tgt_pow_sound] are now PROVED (no longer admitted).
    Their proofs appear further below, after the [iter_src_cong] /
    [iter_tgt_cong] congruence lemmas that they depend on. *)

(** idext is id^c of the corresponding pow – pure arithmetic on toTerm. *)
Lemma toTerm_idext_src : forall c u,
  toTerm (idext_src c u) = iter_id c (toTerm (src_pow c u)).
Proof.
  intros c u. unfold idext_src, toTerm. cbn [nl ne nm na].
  rewrite (Nat.add_comm (nl (src_pow c u)) c). rewrite iter_id_add. reflexivity.
Qed.

Lemma toTerm_idext_tgt : forall c u,
  toTerm (idext_tgt c u) = iter_id c (toTerm (tgt_pow c u)).
Proof.
  intros c u. unfold idext_tgt, toTerm. cbn [nl ne nm na].
  rewrite (Nat.add_comm (nl (tgt_pow c u)) c). rewrite iter_id_add. reflexivity.
Qed.

(** ** Well-formedness / canonicity of [src_pow]/[tgt_pow]. *)

Lemma src_pow_nwf : forall c u, nwf u -> c <= ndim u -> nwf (src_pow c u).
Proof.
  intros c u Hn Hd. unfold src_pow. destruct (c <=? nl u) eqn:E.

```

```

- unfold nwf in *; cbn [nm na]. exact Hn.
- apply Nat.leb_gt in E. unfold nwf, ndim in *; cbn [nm na]. lia.
Qed.

Lemma tgt_pow_nwf : forall c u, nwf u -> c <= ndim u -> nwf (tgt_pow c u).
Proof.
  intros c u Hn Hd. unfold tgt_pow. destruct (c <=? nl u) eqn:E.
  - unfold nwf in *; cbn [nm na]. exact Hn.
  - apply Nat.leb_gt in E. unfold nwf, ndim in *; cbn [nm na]. lia.
Qed.

Lemma src_pow_canonical : forall c u, canonical u -> canonical (src_pow c u).
Proof.
  intros c u Hc. unfold src_pow. destruct (c <=? nl u) eqn:E.
  - unfold canonical in *; cbn [nm ne]. exact Hc.
  - unfold canonical; cbn [ne]. intros _; reflexivity.
Qed.

Lemma tgt_pow_canonical : forall c u, canonical u -> canonical (tgt_pow c u).
Proof.
  intros c u Hc. unfold tgt_pow. destruct (c <=? nl u) eqn:E.
  - unfold canonical in *; cbn [nm ne]. exact Hc.
  - apply Nat.leb_gt in E. unfold canonical; cbn [nm ne]. intro H0; lia.
Qed.

(** ** Iterated src/tgt preserve well-formedness and are congruences. *)

Lemma wf_iter_src : forall c x d, wf x d -> c <= d -> wf (iter_src c x) (d - c).
Proof.
  induction c; intros x d Hx Hc.
  - rewrite Nat.sub_0_r. exact Hx.
  - change (iter_src (S c) x) with (src (iter_src c x)).
    apply (wf_src (iter_src c x) (d - S c)).
    replace (S (d - S c)) with (d - c) by lia.
    apply IHc; [exact Hx | lia].
Qed.

Lemma wf_iter_tgt : forall c x d, wf x d -> c <= d -> wf (iter_tgt c x) (d - c).
Proof.
  induction c; intros x d Hx Hc.
  - rewrite Nat.sub_0_r. exact Hx.
  - change (iter_tgt (S c) x) with (tgt (iter_tgt c x)).
    apply (wf_tgt (iter_tgt c x) (d - S c)).
    replace (S (d - S c)) with (d - c) by lia.
    apply IHc; [exact Hx | lia].
Qed.

Lemma iter_src_cong : forall c x y d,
  x == y -> wf x d -> wf y d -> c <= d -> iter_src c x == iter_src c y.
Proof.
  induction c; intros x y d Hxy Hx Hy Hc.
  - exact Hxy.
  - change (iter_src (S c) x) with (src (iter_src c x)).
    change (iter_src (S c) y) with (src (iter_src c y)).

```

```

    apply (E5_src (iter_src c x) (iter_src c y) (d - S c)).
+   apply (IHc x y d); [exact Hxy | exact Hx | exact Hy | lia].
+   replace (S (d - S c)) with (d - c) by lia. apply wf_iter_src; [exact Hx | lia].
+   replace (S (d - S c)) with (d - c) by lia. apply wf_iter_src; [exact Hy | lia].

```

Qed.

```

Lemma iter_tgt_cong : forall c x y d,
  x == y -> wf x d -> wf y d -> c <= d -> iter_tgt c x == iter_tgt c y.

```

Proof.

```

  induction c; intros x y d Hxy Hx Hy Hc.
-   exact Hxy.
-   change (iter_tgt (S c) x) with (tgt (iter_tgt c x)).
    change (iter_tgt (S c) y) with (tgt (iter_tgt c y)).
    apply (E6_tgt (iter_tgt c x) (iter_tgt c y) (d - S c)).
+   apply (IHc x y d); [exact Hxy | exact Hx | exact Hy | lia].
+   replace (S (d - S c)) with (d - c) by lia. apply wf_iter_tgt; [exact Hx | lia].
+   replace (S (d - S c)) with (d - c) by lia. apply wf_iter_tgt; [exact Hy | lia].

```

Qed.

*(** ** Commutation lemmas: NF-level [src_pow]/[tgt_pow] match the term-level iterated [src]/[tgt]. These are [src_pow_sound] / [tgt_pow_sound], two of the formerly-admitted lemmas, now proved. NF-only, no circularity: they use only the structural axioms (IC1-IC4) and the congruence / well-formedness lemmas above. *)*

*(** Peel one application from the INSIDE of an iteration (the stdlib only provides the outside peel [Nat.iter_succ]). *)*

```

Lemma iter_inside : forall (f : Term -> Term) c t,
  Nat.iter (S c) f t = Nat.iter c f (f t).

```

Proof.

```

  intros f. induction c as [|c' IH]; intro t;
  [reflexivity | simpl; rewrite <- IH; reflexivity].

```

Qed.

*(** Cancellation: [c] sources strip [c] identities pairwise (each pair killed by IC3, [src (id x) == x]). No dimension bound needed. *)*

```

Lemma iter_src_id_cancel : forall c Y d, wf Y d -> iter_src c (iter_id c Y) == Y.

```

Proof.

```

  induction c as [|c' IH]; intros Y d HY.
-   simpl. apply E1_refl.
-   rewrite (iter_id_S c' Y).
    unfold iter_src. rewrite (iter_inside src c' (id (iter_id c' Y))).
    set (W := iter_id c' Y).
    assert (HW : wf W (d + c')) by (apply (wf_iter_id c' HY)).
    apply (E3_trans (Nat.iter c' src (src (id W))) (Nat.iter c' src W) Y).
+   apply (iter_src_cong c' (src (id W)) W (d + c')).
    * apply (IC3 W (d + c') HW).
    * apply (wf_src (id W) (d + c')). apply (wf_id W (d + c') HW).
    * exact HW.
    * lia.
+   unfold W. exact (IH Y d HY).

```

Qed.

*(** Build a src-block: [S j] sources land on an end-block as a single [Es]-block*

```

    (iterating [src_end_collapse], driven by IC1/IC2). *)
Lemma iter_src_end_build : forall j m e a,
  S m + j <= gen_dim a ->
  iter_src (S j) (iter_end e m (gen a)) == iter_end Es (S m + j) (gen a).
Proof.
  induction j as [|j' IH]; intros m e a Hb.
  - unfold iter_src; simpl. rewrite Nat.add_0_r. apply (src_end_collapse e). lia.
  - unfold iter_src. rewrite (iter_inside_src (S j') (iter_end e m (gen a))).
    apply (E3_trans (Nat.iter (S j') src (src (iter_end e m (gen a))))
      (Nat.iter (S j') src (iter_end Es (S m) (gen a)))
      (iter_end Es (S m + S j') (gen a))).
  + apply (iter_src_cong (S j') (src (iter_end e m (gen a)))
    (iter_end Es (S m) (gen a)) (gen_dim a - S m)).
    * apply (src_end_collapse e). lia.
    * apply (wf_src (iter_end e m (gen a)) (gen_dim a - S m)).
      replace (S (gen_dim a - S m)) with (gen_dim a - m) by lia.
      apply (wf_iter_end e). lia.
    * apply (wf_iter_end Es). lia.
    * lia.
  + change (Nat.iter (S j') src (iter_end Es (S m) (gen a)))
    with (iter_src (S j') (iter_end Es (S m) (gen a))).
    replace (S m + S j') with (S (S m) + j') by lia. apply IH. lia.
Qed.

(** [src_pow_sound]: the NF-level iterated source is faithful to the term-level
    one. Case split mirrors [src_pow]: enough identities to strip (cancellation)
    vs. not (cancel all ids, then build a src-block). *)
Lemma src_pow_sound : forall c u,
  nwf u -> c <= ndim u -> toTerm (src_pow c u) == iter_src c (toTerm u).
Proof.
  intros c [l e m a] Hnwf Hdim.
  unfold nwf in Hnwf; cbn [na nm] in Hnwf.
  unfold ndim in Hdim; cbn [nl na nm] in Hdim.
  set (B := iter_end e m (gen a)).
  assert (HB : wf B (gen_dim a - m)) by (unfold B; apply (wf_iter_end e); exact Hnwf).
  unfold src_pow; cbn [nl ne nm na].
  destruct (le_gt_dec c l) as [Hle | Hgt].
  - assert (E : (c <=? l) = true) by (apply Nat.leb_le; exact Hle).
    rewrite E. unfold toTerm; cbn [nl ne nm na]. fold B.
    apply E2_sym.
    replace (iter_id l B) with (iter_id c (iter_id (l - c) B))
      by (rewrite <- iter_id_add; f_equal; lia).
    apply (iter_src_id_cancel c (iter_id (l - c) B) ((gen_dim a - m) + (l - c))).
    apply (wf_iter_id (l - c) HB).
  - assert (E : (c <=? l) = false) by (apply Nat.leb_gt; exact Hgt).
    rewrite E. unfold toTerm; cbn [nl ne nm na iter_id]. fold B.
    assert (Hsplit : iter_src c (iter_id l B)
      = iter_src (c - l) (iter_src l (iter_id l B)))
      by (rewrite <- iter_src_add; f_equal; lia).
    assert (Hmain : iter_src c (iter_id l B) == iter_end Es (m + (c - l)) (gen a)).
    { rewrite Hsplit.
      apply (E3_trans (iter_src (c - l) (iter_src l (iter_id l B)))
        (iter_src (c - l) B)
        (iter_end Es (m + (c - l)) (gen a))).

```

```

- apply (iter_src_cong (c - l) (iter_src l (iter_id l B)) B (gen_dim a - m)).
+ apply (iter_src_id_cancel l B (gen_dim a - m) HB).
+ replace (gen_dim a - m) with (((gen_dim a - m) + l) - l) by lia.
  apply (wf_iter_src l (iter_id l B) ((gen_dim a - m) + l));
  [apply (wf_iter_id l HB) | lia].
+ exact HB.
+ lia.
- destruct (c - l) as [|j'] eqn:Hcl; [lia|].
  unfold B. replace (m + S j') with (S m + j') by lia.
  apply (iter_src_end_build j' m e a). lia. }
exact (E2_sym _ _ Hmain).
Qed.

```

*(** Mirror of [iter_src_id_cancel] (uses IC4 instead of IC3). *)*

Lemma iter_tgt_id_cancel : forall c Y d, wf Y d -> iter_tgt c (iter_id c Y) == Y.

Proof.

```

induction c as [|c' IH]; intros Y d HY.
- simpl. apply E1_refl.
- rewrite (iter_id_S c' Y).
  unfold iter_tgt. rewrite (iter_inside tgt c' (id (iter_id c' Y))).
  set (W := iter_id c' Y).
  assert (HW : wf W (d + c')) by (apply (wf_iter_id c' HY)).
  apply (E3_trans (Nat.iter c' tgt (tgt (id W))) (Nat.iter c' tgt W) Y).
+ apply (iter_tgt_cong c' (tgt (id W)) W (d + c')).
  * apply (IC4 W (d + c') HW).
  * apply (wf_tgt (id W) (d + c')). apply (wf_id W (d + c') HW).
  * exact HW.
  * lia.
+ unfold W. exact (IH Y d HY).

```

Qed.

*(** Mirror of [iter_src_end_build] (uses [tgt_end_collapse]). *)*

Lemma iter_tgt_end_build : forall j m e a,

S m + j <= gen_dim a ->

iter_tgt (S j) (iter_end e m (gen a)) == iter_end Et (S m + j) (gen a).

Proof.

```

induction j as [|j' IH]; intros m e a Hb.
- unfold iter_tgt; simpl. rewrite Nat.add_0_r. apply (tgt_end_collapse e). lia.
- unfold iter_tgt. rewrite (iter_inside tgt (S j') (iter_end e m (gen a))).
  apply (E3_trans (Nat.iter (S j') tgt (tgt (iter_end e m (gen a))))
    (Nat.iter (S j') tgt (iter_end Et (S m) (gen a)))
    (iter_end Et (S m + S j') (gen a))).
+ apply (iter_tgt_cong (S j') (tgt (iter_end e m (gen a)))
  (iter_end Et (S m) (gen a)) (gen_dim a - S m)).
  * apply (tgt_end_collapse e). lia.
  * apply (wf_tgt (iter_end e m (gen a)) (gen_dim a - S m)).
    replace (S (gen_dim a - S m)) with (gen_dim a - m) by lia.
    apply (wf_iter_end e). lia.
  * apply (wf_iter_end Et). lia.
  * lia.
+ change (Nat.iter (S j') tgt (iter_end Et (S m) (gen a)))
  with (iter_tgt (S j') (iter_end Et (S m) (gen a))).
  replace (S m + S j') with (S (S m) + j') by lia. apply IH. lia.

```

Qed.

```

(** [tgt_pow_sound]: mirror of [src_pow_sound]. *)
Lemma tgt_pow_sound : forall c u,
  nwf u -> c <= ndim u -> toTerm (tgt_pow c u) == iter_tgt c (toTerm u).
Proof.
  intros c [l e m a] Hnwf Hdim.
  unfold nwf in Hnwf; cbn [na nm] in Hnwf.
  unfold ndim in Hdim; cbn [nl na nm] in Hdim.
  set (B := iter_end e m (gen a)).
  assert (HB : wf B (gen_dim a - m)) by (unfold B; apply (wf_iter_end e); exact Hnwf).
  unfold tgt_pow; cbn [nl ne nm na].
  destruct (le_gt_dec c l) as [Hle | Hgt].
  - assert (E : (c <= l) = true) by (apply Nat.leb_le; exact Hle).
    rewrite E. unfold toTerm; cbn [nl ne nm na]. fold B.
    apply E2_sym.
    replace (iter_id l B) with (iter_id c (iter_id (l - c) B))
      by (rewrite <- iter_id_add; f_equal; lia).
    apply (iter_tgt_id_cancel c (iter_id (l - c) B) ((gen_dim a - m) + (l - c))).
    apply (wf_iter_id (l - c) HB).
  - assert (E : (c <= l) = false) by (apply Nat.leb_gt; exact Hgt).
    rewrite E. unfold toTerm; cbn [nl ne nm na iter_id]. fold B.
    assert (Hsplit : iter_tgt c (iter_id l B)
      = iter_tgt (c - l) (iter_tgt l (iter_id l B)))
      by (rewrite <- iter_tgt_add; f_equal; lia).
    assert (Hmain : iter_tgt c (iter_id l B) == iter_end Et (m + (c - l)) (gen a)).
    { rewrite Hsplit.
      apply (E3_trans (iter_tgt (c - l) (iter_tgt l (iter_id l B)))
        (iter_tgt (c - l) B)
        (iter_end Et (m + (c - l)) (gen a))).
      - apply (iter_tgt_cong (c - l) (iter_tgt l (iter_id l B)) B (gen_dim a - m)).
        + apply (iter_tgt_id_cancel l B (gen_dim a - m) HB).
        + replace (gen_dim a - m) with (((gen_dim a - m) + l) - l) by lia.
          apply (wf_iter_tgt l (iter_id l B) ((gen_dim a - m) + l));
            [ apply (wf_iter_id l HB) | lia ].
        + exact HB.
        + lia.
      - destruct (c - l) as [|j'|] eqn:Hcl; [lia].
        unfold B. replace (m + S j') with (S m + j') by lia.
        apply (iter_tgt_end_build j' m e a). lia. }
    exact (E2_sym _ _ Hmain).
Qed.

(** ** Soundness of the normalizer (independent of injectivity).
  NOW PROVED as a corollary of the strengthened [normalize_total] below
  (which carries the soundness conjunct [toTerm u == t]); the definition
  appears immediately after [normalize_total]. *)

(** ** Lemma 4.2 (injectivity of the normal-form interpretation).
  THE deep lemma: distinct canonical, well-formed normal forms denote
  distinct cells of the free model. Discharged by the total semantic
  interpretation of [Model.v] ([Model.toTerm_inj], proved by the combined
  [eq_cat]/[wf] mutual induction in [Model.interp_sound_mut]). *)
Theorem toTerm_inj : forall u v,
  nwf u -> canonical u -> nwf v -> canonical v ->

```



```

toTerm u == toTerm v -> u = v.
Proof. exact Model.toTerm_inj. Qed.

(** Iterated [id] is a congruence (E8_id carries no [wf] premise). *)
Lemma iter_id_cong : forall c x y, x == y -> iter_id c x == iter_id c y.
Proof.
  induction c as [|c' IH]; intros x y H.
  - exact H.
  - rewrite !iter_id_S. apply E8_id. apply IH. exact H.
Qed.

(** ** Lemma 4.4 combinatorial step: from a matched boundary (as NF equality)
    the composite is degenerate and [nf_comp] succeeds with a well-formed,
    canonical result that denotes the composite.

    NOW PROVED (no longer admitted). Two extra hypotheses [toTerm u == a] and
    [toTerm v == b] are taken as inputs: these are exactly the inductive
    hypotheses available at the sole call site ([normalize_total]), and taking
    them here keeps this lemma INDEPENDENT of [normalize_sound] (whose own
    [comp] case is what calls this lemma – so making it depend on
    [normalize_sound] would be circular).

    The matched boundary forces, via [m_comp_boundary], one of two unit
    patterns: either [v] is the right identity-extension of [u] (so the
    composite collapses to [a] by IC11), or [u] is the left identity-extension
    of [v] (collapse to [b] by IC12). In each case [nf_comp] returns the
    surviving operand and [E7_comp] + the unit law identify it with
    [comp k a b]. *)
Lemma nf_comp_glue : forall k a b n u v,
  wf a n -> wf b n -> k < n ->
  normalize a = Some u -> normalize b = Some v ->
  nwf u -> canonical u -> ndim u = n ->
  nwf v -> canonical v -> ndim v = n ->
  toTerm u == a -> toTerm v == b ->
  src_pow (n - k) u = tgt_pow (n - k) v ->
  exists w, nf_comp k u v = Some w /\ nwf w /\ canonical w /\ ndim w = n
  /\ toTerm w == comp k a b.
Proof.
  intros k a b n u v Ha Hb Hk Hna Hnb Hnu Hcu Hdu Hnv Hcv Hdv Hsu Hsv Hglue.
  set (c := n - k) in *.
  assert (Hcle : c <= n) by (unfold c; lia).
  assert (Hspa : toTerm (src_pow c u) == iter_src c a).
  { apply (E3_trans _ (iter_src c (toTerm u)) _).
    - apply src_pow_sound; [exact Hnu | rewrite Hdu; exact Hcle].
    - apply (iter_src_cong c (toTerm u) a n);
      [exact Hsu | rewrite <- Hdu; apply toTerm_wf; exact Hnu | exact Ha | lia]. }
  assert (Htpb : toTerm (tgt_pow c v) == iter_tgt c b).
  { apply (E3_trans _ (iter_tgt c (toTerm v)) _).
    - apply tgt_pow_sound; [exact Hnv | rewrite Hdv; exact Hcle].
    - apply (iter_tgt_cong c (toTerm v) b n);
      [exact Hsv | rewrite <- Hdv; apply toTerm_wf; exact Hnv | exact Hb | lia]. }
  assert (Hbnd : iter_src c a == iter_tgt c b).
  { apply (E3_trans _ (toTerm (src_pow c u)) _).
    - apply E2_sym; exact Hspa.

```

```

- rewrite Hglue; exact Htpb. }
assert (Hwfcomp : wf (comp k a b) n).
{ apply (wf_comp k a b n Ha Hb Hk). unfold c in Hbnd. exact Hbnd. }
assert (Hnf : nf_comp k u v =
  (if NF_eqb v (idext_src c u) then Some u
   else if NF_eqb u (idext_tgt c v) then Some v else None)).
{ unfold nf_comp; cbv zeta. rewrite Hdu, Hdvd, Nat.eqb_refl; simpl negb.
  assert (Hkb : (k <? n) = true) by (apply Nat.ltb_lt; exact Hk).
  rewrite Hkb; simpl negb. unfold c. reflexivity. }
destruct (NF_eqb v (idext_src c u)) eqn:E1.
- (* right unit IC11: result u *)
  exists u.
  split; [rewrite Hnf; reflexivity|].
  split; [exact Hnu|]. split; [exact Hcu|]. split; [exact Hdu|].
  assert (Hv : v = idext_src c u)
    by (apply NF_eqb_eq; [exact Hcv | apply idext_src_canonical; exact Hcu | exact
      ↪ E1]).
  assert (Hb2 : b == iter_id c (iter_src c a)).
  { apply (E3_trans _ (toTerm v) _).
    - apply E2_sym; exact Hsv.
    - rewrite Hv, toTerm_idext_src. apply iter_id_cong; exact Hspa. }
  assert (Hwfr : wf (comp k a (iter_id c (iter_src c a))) n).
  { apply (wf_comp k a (iter_id c (iter_src c a)) n Ha).
    - replace n with ((n - c) + c) by lia.
      apply wf_iter_id. apply (wf_iter_src c a n); [exact Ha | exact Hcle].
    - exact Hk.
    - unfold c. apply E2_sym.
      apply (iter_tgt_id_cancel (n - k) (iter_src (n - k) a) (n - (n - k))).
      apply (wf_iter_src (n - k) a n); [exact Ha | lia]. }
  assert (HIC11 : comp k a (iter_id c (iter_src c a)) == a).
  { unfold c. apply (IC11 a k n Ha); [lia | unfold c in Hwfr; exact Hwfr]. }
  apply (E3_trans _ a _); [exact Hsu|]. apply E2_sym.
  apply (E3_trans _ (comp k a (iter_id c (iter_src c a))) _).
  + apply (E7_comp a a b (iter_id c (iter_src c a)) k n);
    [apply E1_refl | exact Hb2 | exact Hwfcomp | exact Hwfr].
  + exact HIC11.
- destruct (NF_eqb u (idext_tgt c v)) eqn:E2.
+ (* left unit IC12: result v *)
  exists v.
  split; [rewrite Hnf; reflexivity|].
  split; [exact Hnv|]. split; [exact Hcv|]. split; [exact Hdvd|].
  assert (Hu : u = idext_tgt c v)
    by (apply NF_eqb_eq; [exact Hcu | apply idext_tgt_canonical; exact Hcv | exact
      ↪ E2]).
  assert (Ha2 : a == iter_id c (iter_tgt c b)).
  { apply (E3_trans _ (toTerm u) _).
    - apply E2_sym; exact Hsu.
    - rewrite Hu, toTerm_idext_tgt. apply iter_id_cong; exact Htpb. }
  assert (Hwfl : wf (comp k (iter_id c (iter_tgt c b)) b) n).
  { apply (wf_comp k (iter_id c (iter_tgt c b)) b n).
    - replace n with ((n - c) + c) by lia.
      apply wf_iter_id. apply (wf_iter_tgt c b n); [exact Hb | exact Hcle].
    - exact Hb.
    - exact Hk.

```

```

- unfold c.
  apply (iter_src_id_cancel (n - k) (iter_tgt (n - k) b) (n - (n - k))).
  apply (wf_iter_tgt (n - k) b n); [exact Hb | lia]. }
assert (HIC12 : comp k (iter_id c (iter_tgt c b)) b == b).
{ unfold c. apply (IC12 b k n Hb); [lia | unfold c in Hwfl; exact Hwfl]. }
apply (E3_trans _ b _); [exact Hsv|]. apply E2_sym.
apply (E3_trans _ (comp k (iter_id c (iter_tgt c b)) b) _).
* apply (E7_comp a (iter_id c (iter_tgt c b)) b b k n);
  [exact Ha2 | apply E1_refl | exact Hwfcomp | exact Hwfl].
* exact HIC12.
+ (* None: impossible by the boundary *)
  exfalso. destruct (m_comp_boundary c u v Hglue) as [HA | HB].
  * rewrite HA, NF_eqb_refl in E1; discriminate.
  * rewrite HB, NF_eqb_refl in E2; discriminate.

```

Qed.

*(** ** Totality AND soundness of the normalizer on well-formed terms.
 The conclusion carries [toTerm u == t]: proving totality and soundness by
 the same [wf] induction is what makes the [comp] case independent of a
 free-standing [normalize_sound] (the operand soundness facts [Hsu]/[Hsv]
 that feed [nf_comp_glue] are exactly the induction hypotheses here). *)*

Theorem normalize_total : forall t d,
 wf t d ->
 exists u, normalize t = Some u /\ nwf u /\ canonical u /\ ndim u = d
 /\ toTerm u == t.

Proof.

```

intros t d H. induction H.
- (* gen *)
  exists (mkNF 0 Es 0 g).
  split; [reflexivity|].
  split; [unfold nwf; cbn; lia|].
  split; [unfold canonical; cbn; reflexivity|].
  split; [unfold ndim; cbn; lia|].
  unfold toTerm; cbn [nl ne nm na]. apply E1_refl.
- (* src e, wf e (S n) *)
  destruct IHwf as (u & Hu & Hnu & Hcu & Hdu & Hsu).
  assert (Hd : ndim u > 0) by lia.
  destruct (nf_src_spec Hnu Hd) as (w & Hsw & Hnw & Hdw & Htw).
  exists w. rewrite normalize_src_eq, Hu, Hsw.
  split; [reflexivity|].
  split; [exact Hnw|].
  split; [apply (nf_src_canonical _ _ Hcu Hsw)|].
  split; [lia|].
  (* toTerm w == src e : peel src via [Htw], then E5_src on [Hsu]. *)
  apply (E3_trans (toTerm w) (src (toTerm u)) (src e)); [exact Htw|].
  apply (E5_src (toTerm u) e n).
  + exact Hsu.
  + rewrite <- Hdu; apply toTerm_wf; exact Hnu.
  + exact H.
- (* tgt e *)
  destruct IHwf as (u & Hu & Hnu & Hcu & Hdu & Hsu).
  assert (Hd : ndim u > 0) by lia.
  destruct (nf_tgt_spec Hnu Hd) as (w & Hsw & Hnw & Hdw & Htw).
  exists w. rewrite normalize_tgt_eq, Hu, Hsw.

```

```

split; [reflexivity]].
split; [exact Hnw]].
split; [apply (nf_tgt_canonical _ _ Hcu Hsw)].
split; [lia]].
apply (E3_trans (toTerm w) (tgt (toTerm u)) (tgt e)); [exact Htw]].
apply (E6_tgt (toTerm u) e n).
+ exact Hsu.
+ rewrite <- Hdu; apply toTerm_wf; exact Hnu.
+ exact H.
- (* id e *)
destruct IHwf as (u & Hu & Hnu & Hcu & Hdu & Hsu).
destruct (nf_id_spec Hnu) as (Hnw & Hdw & Htw).
exists (nf_id u). rewrite normalize_id_eq, Hu.
split; [reflexivity]].
split; [exact Hnw]].
split; [unfold canonical, nf_id; cbn [nl ne nm na]; exact Hcu]].
split; [lia]].
apply (E3_trans (toTerm (nf_id u)) (id (toTerm u)) (id e)); [exact Htw]].
apply E8_id; exact Hsu.
- (* comp k a b *)
destruct IHwf1 as (u & Hu & Hnu & Hcu & Hdu & Hsu).
destruct IHwf2 as (v & Hv & Hnv & Hcv & Hdv & Hsv).
(* [Hsu : toTerm u == a] and [Hsv : toTerm v == b] now come directly from
the induction hypotheses – no appeal to a free-standing soundness lemma,
so totality and soundness are established by this single induction. *)
assert (Hwu : wf (toTerm u) n) by (rewrite <- Hdu; apply toTerm_wf; exact Hnu).
assert (Hwv : wf (toTerm v) n) by (rewrite <- Hdv; apply toTerm_wf; exact Hnv).
(* boundary, via soundness + injectivity, yields src_pow = tgt_pow *)
assert (Hglue : src_pow (n - k) u = tgt_pow (n - k) v).
{ apply toTerm_inj.
- apply src_pow_nwf; [exact Hnu | lia].
- apply src_pow_canonical; exact Hcu.
- apply tgt_pow_nwf; [exact Hnv | lia].
- apply tgt_pow_canonical; exact Hcv.
- (* toTerm (src_pow c u) == toTerm (tgt_pow c v) *)
  apply (E3_trans (toTerm (src_pow (n - k) u))
    (iter_src (n - k) a)
    (toTerm (tgt_pow (n - k) v))).
+ apply (E3_trans (toTerm (src_pow (n - k) u))
  (iter_src (n - k) (toTerm u))
  (iter_src (n - k) a)).
  * apply src_pow_sound; [exact Hnu | lia].
  * apply (iter_src_cong (n - k) (toTerm u) a n);
    [exact Hsu | exact Hwu | exact H | lia].
+ apply (E3_trans (iter_src (n - k) a)
  (iter_tgt (n - k) b)
  (toTerm (tgt_pow (n - k) v))).
  * exact H2. (* the boundary premise *)
  * apply (E3_trans (iter_tgt (n - k) b)
    (iter_tgt (n - k) (toTerm v))
    (toTerm (tgt_pow (n - k) v))).
  -- apply (iter_tgt_cong (n - k) b (toTerm v) n);
    [apply E2_sym; exact Hsv | exact H0 | exact Hwv | lia].
  -- apply E2_sym; apply tgt_pow_sound; [exact Hnv | lia]. }

```

```

    destruct (nf_comp_glue _ _ _ _ _ H H0 H1 Hu Hv Hnu Hcu Hdu Hnv Hcv Hdv Hsu Hsv
      ↪ Hglue)
      as (w & Hcw & Hnw & Hcanw & Hdw & Htw).
    exists w. rewrite normalize_comp_eq, Hu, Hv. rewrite Hcw.
    split; [reflexivity].
    split; [exact Hnw].
    split; [exact Hcanw].
    split; [exact Hdw].
    exact Htw.
Qed.

Arguments normalize_total {t d}.

(** ** Soundness of the normalizer, as a corollary of [normalize_total]. *)
Theorem normalize_sound : forall t d u,
  wf t d -> normalize t = Some u -> toTerm u == t.
Proof.
  intros t d u Hwf Hn.
  destruct (normalize_total Hwf) as (u' & Hu' & _ & _ & Hsu').
  rewrite Hn in Hu'. injection Hu' as -. exact Hsu'.
Qed.
Arguments normalize_sound {t d u}.

(** ** Completeness: provable equality implies identical normal forms. *)
Theorem normalize_complete : forall t1 t2 d1 d2,
  wf t1 d1 -> wf t2 d2 -> t1 == t2 -> normalize t1 = normalize t2.
Proof.
  intros t1 t2 d1 d2 H1 H2 Heq.
  destruct (normalize_total H1) as (u & Hu & Hnu & Hcu & Hdu & Hsu).
  destruct (normalize_total H2) as (v & Hv & Hnv & Hcv & Hdv & Hsv).
  rewrite Hu, Hv. f_equal.
  apply toTerm_inj; try assumption.
  apply (E3_trans (toTerm u) t1 (toTerm v)).
  - apply (normalize_sound H1 Hu).
  - apply (E3_trans t1 t2 (toTerm v)).
    + exact Heq.
    + apply E2_sym. apply (normalize_sound H2 Hv).
Qed.

Arguments normalize_complete {t1 t2 d1 d2}.

(** ** Theorem 4.6: decidability of equality in the strict free model. *)
Theorem free_cell_decidable : forall c1 c2 : FreeCell, {c1 == c2} + {~ c1 == c2}.
Proof.
  intros [t1 d1 p1] [t2 d2 p2]. unfold FreeCell_eq; simpl.
  (* Compute normal forms directly; totality rules out the None branches. *)
  destruct (normalize t1) as [u|] eqn:Hu.
  2:{ ex falso. destruct (normalize_total p1) as (u & Hu' & _).
    rewrite Hu in Hu'; discriminate. }
  destruct (normalize t2) as [v|] eqn:Hv.
  2:{ ex falso. destruct (normalize_total p2) as (v & Hv' & _).
    rewrite Hv in Hv'; discriminate. }
  destruct (NF_eq_dec u v) as [E | N].
  - left.

```

```

    apply (E3_trans t1 (toTerm u) t2).
+   apply E2_sym; apply (normalize_sound p1 Hu).
+   subst v. apply (normalize_sound p2 Hv).
-   right. intro Hc. apply N.
    assert (Hnn : normalize t1 = normalize t2)
      by (apply (normalize_complete p1 p2 Hc)).
    rewrite Hu, Hv in Hnn. injection Hnn; auto.
Qed.

```